
How Hard Can It Be? Hardness-Aware Multi-Objective Unlearning

Jiangwei Chen^{*12} Xinyuan Niu^{*13} Rachael Hwee Ling Sim¹ Zhengyuan Liu² Nancy F. Chen²³
Bryan Kian Hsiang Low¹

Abstract

Machine unlearning aims to remove the influence of specific forget training data due to privacy, copyright or bias concerns while maintaining the model performance on the remaining retain data. Existing unlearning algorithms, such as optimizing a weighted combination of losses, have tried to achieve these objectives of improving forget quality and maintaining retain utility. However, they do not guarantee that these objectives can be improved by a specified extent for all forget and retain data. In this work, we address this limitation with a novel and theoretically-grounded approach from a constrained optimization perspective. Firstly, we identify that the *hardness* of reconciling both objectives can be quantified by the similarity between the forget data and the retain data. Next, we derive an unlearning algorithm (HAMU) with the overall goal of guaranteeing a specified improvement in forget quality while minimizing the retain utility cost/degradation by updating the model weights based on our hardness measure. Our hardness measure also informs users when retain utility degradation is unavoidable, i.e., both objectives cannot be improved simultaneously, and stopping should be considered. Our algorithm is applicable to non-convex models and is easily parallelizable, making it readily deployable in real-world scenarios. We empirically demonstrate HAMU’s superior performance over baselines on both image and text datasets using large models. Our code is available at <https://github.com/aoi3142/HAMU>.

1. Introduction

Machine unlearning (Cao & Yang, 2015) presents an appealing paradigm to combat the data issues arising from the prevalent adoption of large *machine learning* (ML) models (Raza et al., 2025; Khan et al., 2025), such as privacy leakage (Pawelczyk et al., 2024), copyright infringement (Eldan & Russinovich, 2023) and harmful bias (Wu et al., 2025). Machine unlearning aims to fulfill both objectives: (1) improve *forget quality* by removing the influence of some forget data, from the ML models; while (2) maintaining *retain utility*, the ML model performance on the remaining retain data. Unlearning usually achieves these two objectives by increasing the loss on the forget data (Zhang et al., 2024) or approximating the retrained model which minimizes the loss on the retain data (Neel et al., 2021).

Most existing unlearning algorithms have neglected the possible conflict between both objectives. To achieve (1), unlearning may incur some *cost of forgetting*, which is the negative impact on the ML model performance (retain utility). For example, it may lead to catastrophic forgetting where the unlearned model performs considerably worse than the retrained model (Nguyen et al., 2020; Fan et al., 2025) or more generally retain utility degradation where the ML model performance or knowledge of the remaining retain data worsens, sacrificing (2). We term this phenomenon of improving forget quality causing retain utility degradation as *collateral forgetting*. Existing works have considered multi-objective unlearning (Jin et al., 2025) that attempts to simultaneously improve both objectives, however, it may not be feasible to do so for all possible forget and retain data. An extreme and intuitive example is that collateral forgetting is unavoidable when the forget and retain data are identical. This conflict oversight raises the following fundamental questions: **How much is the cost of forgetting (in terms of retain utility degradation) to improve forget quality by a specified extent? How hard is it to reconcile both objectives in machine unlearning?**

We answer the above questions by quantifying the hardness of unlearning, κ , as a model-dependent continuous measure that depends on the similarity between the forget and retain data. Then, we establish a theoretical connection which shows that the cost of forgetting, in terms of retain

^{*}Equal contribution ¹Department of Computer Science, National University of Singapore, Singapore ²Institute for Infocomm Research, Agency for Science, Technology and Research, Singapore ³Centre for Frontier AI Research, Agency for Science, Technology and Research, Singapore. Correspondence to: Bryan Kian Hsiang Low <lowkh@comp.nus.edu.sg>.

utility degradation, generally increases with the hardness measure (Fig. 1). For example, the cost is the highest in the hardest scenario where the forget and retain data are identical. Moreover, we identify a hardness threshold κ_2 (Fig. 1), whose exceedance informs the model owner prior to unlearning that local collateral forgetting is unavoidable and small updates to improve forget quality will degrade retain utility (i.e., the cost of forgetting is positive).

Next, how can the model owner ensure that the forget quality improves by a specified extent and reduces the cost of forgetting, even for non-convex¹ ML models such as neural networks (NNs)? At first glance, one solution would be to use a carefully tuned weighted combination of the forget and retain losses. However, this simple solution does not allow the model owner to guarantee that the forget quality would increase by a specified extent (Kurmanji et al., 2023; Duan et al., 2025). In contrast, we formulate unlearning in each iteration as a constrained optimization problem where the goal is to minimize the cost of forgetting (the first-order approximation of the retain utility degradation) while constraining that the linear approximation of the forget quality must improve by the specified extent. Our hardness-aware multi-objective unlearning (HAMU) algorithm decides on the unlearning weight update based on the hardness measure (Fig. 1). For harder scenarios, our HAMU algorithm uses the rectified update which incorporates the forget data’s gradients to guarantee forget quality improvement. HAMU is efficient but applicable to non-convex models as we consider the first-order approximation of the loss function limited to a small local radius. The linear and layer-wise independence nature of HAMU at each iteration also makes it scalable and parallelizable for use with *large language models* (LLMs). We empirically show that HAMU outperforms other baselines in terms of unlearning performance and reduces the cost of forgetting through extensive experiments.

To summarize, our key contributions include:

- We introduce a new unlearning perspective — the goal is to improve an objective (e.g., forget quality) by a specified extent while optimizing the other objective (e.g., minimizing the cost of forgetting/retain utility degradation) (Sec. 4).
- We quantify the hardness of unlearning and establish a *theoretical connection* between hardness and cost of forgetting (Sec. 4.1). The hardness measure also determines the unlearning weight update in our HAMU algorithm (Sec. 4.2) and when *collateral forgetting* occurs (Sec. 4.3).

¹Methods with theoretical guarantees usually require strong assumptions such as strongly-convex loss functions (Ullah et al., 2021; Allouah et al., 2025), which limit their applicability to non-convex models.

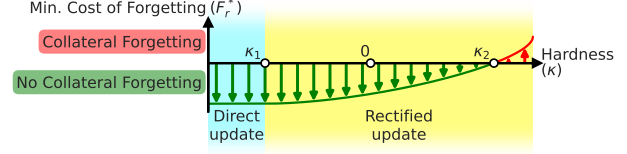


Figure 1. Hardness regimes characterized by the measure κ for fixed $\|g_r\|$ and $\|g_f\|$. Higher κ corresponds to higher cost of forgetting in terms of retain utility degradation. The threshold κ_1 and κ_2 , respectively, determines when the rectified update and unavoidable retain utility degradation (i.e., collateral forgetting) occurs to improve the forget quality by at least Q .

- We adapt HAMU to efficiently and effectively work for large, non-convex ML models (Sec. 5).
- We demonstrate HAMU consistently outperforms baselines across datasets and model architectures through extensive experiments (Sec. 6).

2. Multi-Objective Machine Unlearning

We consider a supervised learning task on the dataset $D \triangleq \{(x_i, y_i)\}_{i=1}^n$, where $y_i \in [K] \triangleq \{1, \dots, K\}$ represents classes or vocabulary tokens. A model (e.g., LLM) parameterized by $w \in \mathbb{R}^d$ is trained or evaluated using the cross-entropy loss $L(p(x; w), y) \triangleq -\log p_y(x; w)$ for each sample (x, y) , where $p(x; w) \in \Delta^{K-1}$ denotes the predicted class probabilities (for classification) or distribution over the next token (for next token prediction). In machine unlearning, the training data D is partitioned into the forget data $D_f \subseteq D$ and the retain data $D_r \triangleq D \setminus D_f$. Let w_0 denote the weights of the original model that is trained to minimize the averaged cross-entropy loss on D . At iteration t (starting with $t = 0$), the unlearning algorithm updates the model weights w_t using information from D_f or D_r , or combination thereof, depending on the objectives, to obtain w_{t+1} .

One common objective in machine unlearning is to approximate the retrained model (Georgiev et al., 2025; Chen & Yang, 2023), i.e., the model trained from scratch on D_r . This is usually achieved by minimizing the retain loss via fine-tuning (e.g., gradient descent on D_r) the original model on the retain data (Neel et al., 2021; Bui et al., 2026), which helps to maintain retain utility. Due to the inadequacy of this objective to ensure forget quality (Block et al., 2025), the objective of increasing forget loss (e.g., gradient ascent on D_f) has been proposed (Halimi et al., 2022; Zhang et al., 2024). However, these gradient-ascent methods may induce retain utility degradation (Mavrothalassitis et al., 2025), making increasing forget loss insufficient as the sole objective (Fan et al., 2025).

Hence, instead of solely approximating the retrained model or increasing the forget loss, we focus on the multi-

objective aspect of unlearning. Existing multi-objective unlearning methods typically optimize a weighted combination of the objectives (Kurmanji et al., 2023; Duan et al., 2025). However, there is no guarantee that the desired performance can be achieved for any of the objectives. Furthermore, they have not considered the inherent statistical dependence between the forget and retain data, hence there is no attempt to reconcile the possibly conflicting objectives. The two objectives of ensuring forget quality and maintaining retain utility are not the same for non-convex models. In fact, we show that depending on the relationship between D_f and D_r , these two objectives can be either conflicting or compatible.

3. Conflict Analysis via Learning Dynamics

We begin by analyzing how taking a single gradient descent step on a sample $z' = (x', y')$ during unlearning affects the loss on another sample $z = (x, y)$. Let $L_t(z) \triangleq L(p(x; w_t), y)$ denote the loss on x at iteration t . We are interested in the quantity $\Delta_{z'} L_t(z) \triangleq L_{t+1}(z) - L_t(z)$ for different choices of z' and z , where $\Delta_{z'}$ means the change in loss is induced by taking a single step on the sample z' . We first consider the simplified case of fine-tuning on a single sample from the retain data. Using first-order approximation², for any $z_r = (x_r, y_r) \in D_r$,

$$\Delta_{z_r} L_t(z_r) = L_{t+1}(z_r) - L_t(z_r) \approx -\eta \cdot \|\nabla_w L_t(z_r)\|^2,$$

where η is the learning rate, and ∇_w denotes the gradient with respect to the model parameters w . Hence, unlearning by fine-tuning on a sample from the retain data reduces the loss on that sample. Now consider how fine-tuning on the same sample impacts the loss on a sample from the forget data. For any $z_f = (x_f, y_f) \in D_f$ we have

$$\Delta_{z_r} L_t(z_f) \approx -\eta \cdot \nabla_w L_t(z_f) \cdot \nabla_w L_t(z_r).$$

Unlike the previous scenario, $\Delta_{z_r} L_t(z_f)$ is not always negative, but depends on the alignment between the two gradients $\nabla_w L_t(z_f)$ and $\nabla_w L_t(z_r)$. When the two **gradients are aligned** (positive dot product), the losses on the forget and retain samples both decrease. When the **gradients are misaligned** (negative dot product), the loss on the forget sample increases as desired. The analysis naturally extends to batch gradients $L_t(D) \triangleq 1/|D| \sum_{z \in D} L_t(z)$. When taking a single gradient step $-\eta \nabla_w L_t(D_r)$ to improve retain utility, the change in the batch forget loss and quality depends on the dot product $\nabla_w L_t(D_f) \cdot \nabla_w L_t(D_r)$. Negative and positive gradient dot product leads to better and worse forget quality, respectively. Intuitively, the dot product fundamentally determines how hard it is to reconcile both objectives of improving retain utility

²We defer the detailed derivation and discussion on why first-order approximation is sufficient to App. B.1.

and forget quality in unlearning. In the next section, we will show that **the gradient dot product also monotonically determines the minimum cost of forgetting, i.e., cost in retain utility to improve forget quality by a specified extent.**

4. First-Order Convex Formulation

Since optimizing for smaller $L_t(D_r)$ and larger $L_t(D_f)$ simultaneously can be conflicting and provides no guarantee for satisfying either objective, we consider the problem of optimizing for the best retain utility while stipulating a minimum requirement on the forget quality, i.e., minimize $L_t(D_r)$ subject to $\Delta L_t(D_f) > Q$, where $Q > 0$ is the per-iteration threshold for the forget quality requirement. Following the first-order approximation in Sec. 3, we focus on the objectives in each iteration. Specifically, at iteration t , the retain utility degradation (i.e., cost of forgetting) $\Delta L_t(D_r) \approx g_r \cdot \Delta w$ and the improvement in forget quality $\Delta L_t(D_f) \approx g_f \cdot \Delta w$, where $g_f \triangleq \nabla_w L_t(D_f)$, $g_r \triangleq \nabla_w L_t(D_r)$ are the averaged gradients, and $\Delta w \triangleq w_{t+1} - w_t$ is the weight update at iteration t . To ensure accurate approximation (see App. B.1), we restrict the per-iteration update to a local neighborhood, i.e., $\|\Delta w\| \leq R$ for some small $R > 0$. Therefore, it is equivalent to solve the following optimization problem³:

$$\begin{aligned} & \underset{\Delta w}{\text{minimize}} && g_r \cdot \Delta w \\ & \text{subject to} && g_f \cdot \Delta w \geq Q, \quad \|\Delta w\| \leq R. \end{aligned} \quad (1)$$

Our formulation is general and allows the use of alternative loss functions such as the NPO loss (Zhang et al., 2024). We state our main results below and defer the full derivations to App. B.

4.1. Hardness of Unlearning

Problem (1) is feasible when $Q \leq R \|g_f\|$, under this feasible condition we can solve the problem analytically. The following theorem provides the closed-form expression for the optimal cost of forgetting in terms of retain utility degradation given a minimum forget quality requirement.

Theorem 4.1. *Let $\kappa \triangleq g_r \cdot g_f$ be the gradient dot product and $g_{r\perp} \triangleq g_r - \frac{g_r \cdot g_f}{\|g_f\|^2} \cdot g_f$ be the component of g_r orthogonal to g_f . Assume⁴ $Q \leq R \|g_f\|$, the optimal value to Problem (1) and the minimum cost of forgetting*

³Unlike multi-objective optimization problems where all objectives are *minimized*, our formulation only minimizes the retain utility degradation and improves the forget quality (i.e., *minimizing* retain loss while *increasing* forget loss) during unlearning.

⁴This assumption is typically satisfied in practice (Sec. 6).

$F_r^* \triangleq \min \mathbf{g}_r \cdot \Delta \mathbf{w}$ is given by

$$\begin{cases} -R \|\mathbf{g}_r\| & \text{if } \kappa \leq \kappa_1 \\ \frac{\kappa Q}{\|\mathbf{g}_f\|^2} - \|\mathbf{g}_{r\perp}\| \sqrt{R^2 - \frac{Q^2}{\|\mathbf{g}_f\|^2}} & \text{otherwise,} \end{cases} \quad (2)$$

where $\kappa_1 \triangleq -Q \|\mathbf{g}_r\| / R$.

As prior hardness measures are largely heuristic and unable to predict unlearning performance for a given instance (Zhao et al., 2024), Theorem 4.1 provides the first theoretical connection between a measurable quantity κ and unlearning objective performance, quantified by the minimum retain utility degradation subject to a sufficient forget quality improvement. Since $\|\mathbf{g}_{r\perp}\|$ also depends on κ , the precise relationship between F_r^* and κ is not immediately apparent. We formalize this relationship below.

Corollary 4.2 (Hardness Measure). *The minimum cost of forgetting F_r^* is monotonically non-decreasing with κ for fixed $\|\mathbf{g}_r\|$ and $\|\mathbf{g}_f\|$.*

Consequently, κ naturally serves as a hardness measure for unlearning. This makes intuitive sense as a larger κ implies $\mathbf{g}_r$ and $\mathbf{g}_f$ are more aligned, and D_r and D_f are more similar. Hence, it is more difficult to reconcile both objectives, i.e., minimizing retain loss while ensuring forget quality, resulting in a higher cost of forgetting and retain utility degradation F_r^* . Consider the extreme example when $D_r = D_f$, it becomes impossible to maintain retain utility while improving forget quality, as the losses on the forget and retain data are identical, making it a particularly hard instance. Note that the hardness measure κ depends not only on the similarity between D_r and D_f , but also on the model itself. This is intuitive, as reconciling the objectives can become more difficult after unlearning the same forget data D_f for several iterations.

4.2. Hardness-Aware Unlearning Update

Interestingly, the hardness measure κ not only indicates the difficulty of the unlearning instance and predicts the cost of forgetting, but also informs the optimal unlearning update, as detailed in the following proposition.

Proposition 4.3 (Optimal Unlearning Update). *Assume $Q \leq R \|\mathbf{g}_f\|$, the minimum cost of forgetting F_r^* is achieved via the optimal unlearning update $\Delta \mathbf{w}^*$, given by*

$$\begin{cases} -\frac{R}{\|\mathbf{g}_r\|} \cdot \mathbf{g}_r & \text{if } \kappa \leq \kappa_1 \\ \frac{Q}{\|\mathbf{g}_f\|^2} \cdot \mathbf{g}_f - \sqrt{R^2 - \frac{Q^2}{\|\mathbf{g}_f\|^2}} \cdot \frac{\mathbf{g}_{r\perp}}{\|\mathbf{g}_{r\perp}\|} & \text{otherwise.} \end{cases} \quad (3)$$

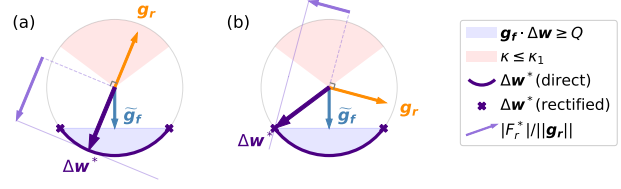


Figure 2. A 2D illustration of HAMU update. Problem (1) restricts the weight update $\Delta \mathbf{w}$ within a circle of radius R and the region where the forget quality constraint is satisfied. In (a), the retain gradient $\mathbf{g}_r$ lies in the “easy” region ($\kappa \leq \kappa_1$), hence the **direct update** is applied, which is in the opposite direction of $\mathbf{g}_r$. In (b), the retain gradient lies outside the “easy” region ($\kappa > \kappa_1$), the **rectified update** is applied, which mixes $\tilde{\mathbf{g}}_f$ and a scaled $\mathbf{g}_{r\perp}$. Here, $\tilde{\mathbf{g}}_f \triangleq (Q / \|\mathbf{g}_f\|^2) \mathbf{g}_f$ and we show all optimal unlearning updates for a fixed $\mathbf{g}_f$ and varying $\mathbf{g}_r$. The resulting change in the retain objective $F_r^* = \mathbf{g}_r \cdot \Delta \mathbf{w}^*$ is also plotted. Note that in both (a) and (b), there is no collateral forgetting ($F_r^* < 0$), and a larger $|F_r^*|$ indicates better resulting retain utility.

The threshold κ_1 determines the level of hardness and governs which type of unlearning update should be applied. When $\kappa \leq \kappa_1$, the objectives are compatible and unlearning is relatively “easy”. It is sufficient to apply the *direct* update in the same direction as $-\mathbf{g}_r$ to improve retain utility without violating the constraint on forget quality since $\mathbf{g}_f \cdot \Delta \mathbf{w}^* \geq Q$ for $\kappa \leq \kappa_1$. Conversely, the objectives become conflicting when $\kappa > \kappa_1$. If we still apply the direct update, the forget quality constraint would be violated as

$$\mathbf{g}_f \cdot \left(-\frac{R}{\|\mathbf{g}_r\|} \cdot \mathbf{g}_r \right) = -\frac{\kappa R}{\|\mathbf{g}_r\|} < -\frac{R}{\|\mathbf{g}_r\|} \cdot \frac{-Q \|\mathbf{g}_r\|}{R} = Q.$$

Thus, when it is “hard” to reconcile the two unlearning objectives, we can no longer move freely in the direction of $-\mathbf{g}_r$, but must instead apply the *rectified* update, which incorporates the direction of $\mathbf{g}_f$ to ensure the forget quality improves by the specified extent Q . In Fig. 2, we illustrate our unlearning updates under different scenarios.

4.3. Unavoidable Collateral Forgetting

Problem (1) only imposes a constraint on the forget quality, and optimizes for the lowest cost of forgetting. However, it is possible that $F_r^* > 0$ and retain utility degrades. This inadvertent erasure of knowledge is also observed in other unlearning works (Nguyen et al., 2020; Tian et al., 2024). We term this phenomenon of improving forget quality causing retain utility degradation as *collateral forgetting*. To identify the per-iteration condition under which collateral forgetting is unavoidable, we investigate an alternative formulation that also imposes a constraint on the retain utility:

$$\mathbf{g}_r \cdot \Delta \mathbf{w} \leq 0, \quad \mathbf{g}_f \cdot \Delta \mathbf{w} \geq Q \quad \text{and} \quad \|\Delta \mathbf{w}\| \leq R. \quad (4)$$

Local collateral forgetting is unavoidable when it is infeasible to find a solution $\Delta \mathbf{w}$ to Problem (4). The following

theorem establishes the exact boundary for this infeasibility.

Theorem 4.4 (Local Collateral Forgetting Condition). *Let $\kappa_2 \triangleq \sqrt{(\|g_r\| \|g_f\|)^2 - Q^2 \|g_r\|^2 / R^2}$. Problem (4) admits no feasible solution for Δw if and only if $\kappa > \kappa_2$, implying that local collateral forgetting is unavoidable.*

The infeasibility condition is again governed by the hardness measure κ . When the problem is too hard ($\kappa > \kappa_2$), it becomes impossible to satisfy the forget quality constraint without compromising the retain utility. By leveraging this condition, model owners can identify unavoidable local collateral forgetting prior to the unlearning process. To better understand this condition, we revisit the extreme example when $D_r = D_f$, in which collateral forgetting is clearly unavoidable. Theorem 4.4 precisely captures this as the hardness κ exceeds the threshold:

$$\kappa = \|g_r\| \|g_f\| > \sqrt{\|g_r\|^2 \left(\|g_f\|^2 - \frac{Q^2}{R^2} \right)} = \kappa_2.$$

The threshold κ_2 provides a clear interpretation. The maximum dot product $\|g_r\| \|g_f\|$ represents the maximum hardness of the problem. The condition implies that if the actual hardness is too close to this maximum, then Problem (4) becomes infeasible. The term $-Q^2 \|g_r\|^2 / R^2$ serves as a safety margin: a more negative value results in a lower threshold κ_2 that is more easily exceeded by κ . For example, when Q is large, we place a stricter requirement on the forget quality, which makes it harder to satisfy both forget and retain constraints simultaneously. The thresholds κ_1 and κ_2 partition the unlearning problem into distinct regimes of hardness, as illustrated in Fig. 1.

5. The HAMU Algorithm

Building on the theoretical analysis in Sec. 4, we propose the *hardness-aware multi-objective unlearning* (HAMU) algorithm. The core of HAMU relies on Proposition 4.3 for the unlearning update and Theorem 4.4 for the stopping criterion. These results imply that after T iterations, our HAMU algorithm can improve the constraint by at least TQ , as long as linear approximation holds.

Stochastic Update. As computing the full gradients g_r and g_f can be computationally expensive, we sample a batch of examples from D_r and D_f at each iteration and use the batch gradient estimates $\bar{g}_r$ and $\bar{g}_f$ to update the weights. The hardness measure (κ) and thresholds (κ_1 , κ_2) are estimated using these batch gradients for the remainder of this section. Based on Proposition 4.3, at each iteration, we first estimate the hardness κ , and then apply either the direct update or the rectified update based on the threshold κ_1 . Following convention in the literature, we use the learning rate

η as our hyperparameter. Specifically, we set $R = \eta \|\bar{g}_r\|$, under this choice of R , the direct update coincides with stochastic gradient descent on D_r . We also investigate the effects of varying batch size in App. G.3 and observe that our method is relatively robust to batch size.

Stopping Criterion. Before applying the unlearning update, we first compare the hardness κ to the threshold κ_2 , and terminate the algorithm if the local collateral forgetting condition is met ($\kappa > \kappa_2$). Theorem 4.4 suggests that the terminated algorithm cannot improve the approximate forget quality by Q without degrading the approximate retain utility. When $Q = 0$, a local approximate Pareto optimum is reached. This convergence to local optima is inherent to first-order methods and is commonly observed in deep learning optimization (Lee et al., 2016). These local minima are typically easy to find and well-performing (Choromanska et al., 2015), as demonstrated by our experiment results. We discuss practical adaptations of our stopping criterion in App. C.

Objective Variants. We propose the HAMU-Q and the HAMU-U algorithms based on two constrained optimization formulations. The former is based on Problem (1) while the latter is based on Problem (16) where we optimize for the best forget quality while requiring the improvement on the retain utility to exceed U (App. B.6). The pseudocode for HAMU-Q and HAMU-U are provided in Alg. 1 and Alg. 2, respectively. Here, Q or U are user-chosen parameters depending on their minimum requirements on forget quality or retain utility, and the only hyperparameter of our algorithms is the learning rate η . We provide guidance on how to select Q and U in Sec. 6.1.

Practical Applicability. In App. B.1, we discuss in mathematical details when HAMU first-order approximations are applicable to complex non-convex models. The approximation error is negligible when the learning rate and the maximum Hessian eigenvalue are small. In App. B.5, we also discuss the second-order formulation which requires a convexity assumption and computation of the Hessian and may be impractical for large ML models. Furthermore, HAMU redoes the approximation at each iteration, *thereby ensuring consistent approximation along the unlearning trajectory*. HAMU only introduces minimal computational overhead (gradient dot product calculation) beyond standard training and is highly parallelizable due to its layer-wise constraint independence (Sec. 5.1). We report the GPU memory and computational cost in App. E.3.

5.1. Parallel and Practical Implementation

Our algorithm scales efficiently to large models by exploiting layer-wise independence, enabling straightforward multi-GPU parallelization. We decompose the flattened weights $w \in \mathbb{R}^d$ into ℓ components corresponding to the

layers of the model, such that $\mathbf{w} = [\mathbf{w}^{(1)}, \dots, \mathbf{w}^{(\ell)}]$, where $\mathbf{w}^{(i)} \in \mathbb{R}^{d_i}$ denotes the weights of layer i . The gradients $\mathbf{g}_f, \mathbf{g}_r$ and the weight update $\Delta \mathbf{w}$ are partitioned similarly. We distribute the global forget quality constraint Q into layer-specific constraints $(Q_i)_{i=1}^\ell$ such that $\sum_{i=1}^\ell Q_i = Q$. Enforcing the local condition $\mathbf{g}_f^{(i)} \cdot \Delta \mathbf{w}^{(i)} \geq Q_i$ guarantees compliance with the global constraint:

$$\mathbf{g}_f \cdot \Delta \mathbf{w} = \sum_{i=1}^\ell \mathbf{g}_f^{(i)} \cdot \Delta \mathbf{w}^{(i)} \geq \sum_{i=1}^\ell Q_i = Q.$$

This allows us to perform layer-wise weight update $\Delta \mathbf{w}^{(i)}$ in Eq. (3) using layer-specific gradients $\mathbf{g}_f^{(i)}$ and $\mathbf{g}_r^{(i)}$. Hence, our algorithm can be efficiently parallelized by assigning different layers to different GPUs. More details about the parallelized implementation can be found in App. G.1. We also discuss how to improve the efficiency of HAMU by using optimizers and practical compatibility of HAMU with modern deep learning frameworks in App. G.

Practitioners have observed that different layers in the same model encode qualitatively distinct information, both for *convolutional neural networks* (CNNs) and LLMs (Raghu et al., 2017; Skeep et al., 2025). Consequently, these layers exhibit varying sensitivities to the same data. To leverage this insight, instead of assigning each layer the same uniform constraint Q/ℓ , we set the layer-wise constraint Q_i to be proportional to the product of the gradient norms,

$$Q_i \triangleq \frac{\|\mathbf{g}_r^{(i)}\| \|\mathbf{g}_f^{(i)}\|}{\sum_{j=1}^\ell \|\mathbf{g}_r^{(j)}\| \|\mathbf{g}_f^{(j)}\|} \cdot Q.$$

Intuitively, a layer i with larger gradient norms is more sensitive to optimization objectives and can contribute more to improving the forget quality, thus, should be assigned a higher Q_i . We demonstrate how this technique further improves unlearning performance in Sec. 6.1.

6. Experiments

In this section, we demonstrate the effectiveness of HAMU and the validity of the hardness measure κ . We compare HAMU-Q and HAMU-U with various unlearning baselines, including fine-tuning on the retain data (FT), gradient ascent on the forget data (GA) (Jang et al., 2023), gradient difference (GDiff) (Liu et al., 2022), KL minimization (KL) (Maini et al., 2024) and SCRUB (Kurmanji et al., 2023). Details on these algorithms can be found in App. D.

We consistently use ΔL_f and $-\Delta L_r$ to denote the change in forget quality (forget loss) and retain utility (negated retain loss). We mainly evaluate the unlearning performance based on the trade-off between ΔL_f and $-\Delta L_r$. Our plots show 5 unlearning epochs, with markers indicating the evaluation at the end of each epoch. The stopping

Algorithm 1 HAMU-Q

```

1: Input: initial weights  $\mathbf{w}_0$ , retain data  $D_r$ , forget data  $D_f$ , batch size  $m$ , learning rate  $\eta$ , maximum iterations  $T$ , forget quality constraint  $Q > 0$ 
2: for  $t = 0, 1, \dots, T - 1$  do
3:   Sample i.i.d. size- $m$  batches  $B_r \sim D_r$  and  $B_f \sim D_f$ 
4:    $\bar{\mathbf{g}}_r \leftarrow \nabla_{\mathbf{w}} L_t(B_r)$ ;  $\bar{\mathbf{g}}_f \leftarrow \nabla_{\mathbf{w}} L_t(B_f)$ 
5:    $R \leftarrow \eta \|\bar{\mathbf{g}}_r\|$ 
6:   if  $Q > R \|\bar{\mathbf{g}}_f\|$  then
7:     break
8:   end if
9:    $\bar{\kappa} \leftarrow \bar{\mathbf{g}}_r \cdot \bar{\mathbf{g}}_f$ 
10:   $\bar{\kappa}_1 \leftarrow -Q \|\bar{\mathbf{g}}_r\| / R$ 
11:   $\bar{\kappa}_2 \leftarrow \sqrt{(\|\bar{\mathbf{g}}_r\| \|\bar{\mathbf{g}}_f\|)^2 - Q^2 \|\bar{\mathbf{g}}_r\|^2 / R^2}$ 
12:  if  $\bar{\kappa} > \bar{\kappa}_2$  then
13:    break
14:  end if
15:  if  $\bar{\kappa} \leq \bar{\kappa}_1$  then
16:     $\Delta \mathbf{w} \leftarrow -\frac{R}{\|\bar{\mathbf{g}}_r\|} \cdot \bar{\mathbf{g}}_r$ 
17:  else
18:     $\bar{\mathbf{g}}_{r\perp} \leftarrow \bar{\mathbf{g}}_r - \frac{\bar{\kappa}}{\|\bar{\mathbf{g}}_f\|^2} \cdot \bar{\mathbf{g}}_f$ 
19:     $\Delta \mathbf{w} \leftarrow \frac{Q}{\|\bar{\mathbf{g}}_f\|^2} \cdot \bar{\mathbf{g}}_f - \sqrt{R^2 - \frac{Q^2}{\|\bar{\mathbf{g}}_f\|^2}} \cdot \frac{\bar{\mathbf{g}}_{r\perp}}{\|\bar{\mathbf{g}}_{r\perp}\|}$ 
20:  end if
21:   $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \Delta \mathbf{w}$ 
22: end for
23: return  $\mathbf{w}_t$ 

```

criterion was not enforced in some of the experiments to illustrate results even for the hardest unlearning settings.

In practice, we observe that the gradient norms $\|\mathbf{g}_r\|$ and $\|\mathbf{g}_f\|$ are usually larger than 1. Hence, we adopt the gradient clipping technique such that $\|\mathbf{g}_r\|_{\max} = \|\mathbf{g}_f\|_{\max} = 1$. We then choose Q such that $Q < \eta \|\mathbf{g}_r\|_{\max} \|\mathbf{g}_f\|_{\max}$. This ensures that the assumption of Theorem 4.1 is generally satisfied in practice. The same is done for U . Unless otherwise stated, our experiments set $Q = U = 0.5\eta$ for HAMU-Q and HAMU-U, respectively. The same learning rate of $1e-4$ is used for all unlearning algorithms.

6.1. Computer Vision Task

We first evaluate HAMU on a *computer vision* (CV) task. To obtain the original model $\mathbf{w}_0$, we randomly initialize a ResNet-20 (He et al., 2016) model and train it on the CIFAR-10 (Krizhevsky, 2009) dataset. Details of the CV unlearning experimental setup are in App. E.

Varying hardness by changing similarity mixing ratio. To construct unlearning problems with different levels of

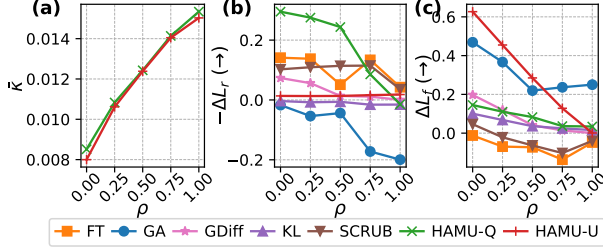


Figure 3. (a) Estimated hardness $\bar{\kappa}$ is strongly correlated with similarity mixing ratio ρ , with Pearson correlation coefficient of 0.994 and 0.986 for HAMU-Q and HAMU-U, respectively. (b, c) Higher ρ (harder) results in a lower increase in $-\Delta L_r$ and ΔL_f .

hardness, we vary the similarity between the forget and retain data using the similarity mixing ratio ρ . Specifically, we first choose a random class c which contains n samples. Given a similarity mixing ratio $\rho \in [0, 1]$, the forget data consists of $n - \lfloor \rho n \rfloor$ randomly sampled data points from class c , and $\lfloor \rho n \rfloor$ randomly sampled data points from the union of all other classes and data points in class c that are not selected in the first step. When $\rho = 0$, the forget data is dissimilar to the retain data and data points from class c appear only in the forget data. In contrast, when $\rho = 1$, the forget data is highly similar to the retain data, and both contain data points from all classes. In our experiments, we use $\rho \in \{0, 0.25, 0.5, 0.75, 1\}$. We use the default $\rho = 0$ in our experiments unless otherwise stated.

Unlearning is harder when the forget and retain data are more similar. Fig. 3a shows that the estimate of our hardness measure, $\bar{\kappa}$, is highly correlated with the similarity mixing ratio ρ . Additionally, Fig. 3b-c shows that as unlearning hardness increases with ρ , the achievable improvements in retain utility $-\Delta L_r$ and forget quality ΔL_f are lower. This verifies our intuition that unlearning becomes harder when forget data is more similar to retain data. Notably, although the hardness measure is derived from HAMU’s formulation, the same trend is consistently observed across other unlearning algorithms (Fig. 3b-c), suggesting that the similarity between the forget and retain data captured by our measure reflects a fundamental characteristic of machine unlearning.

HAMU outperforms baselines, particularly when the problem is hard. In Fig. 4, we compare the unlearning trajectories of HAMU against baselines for unlearning scenarios of different levels of hardness, controlled by the similarity mixing ratio ρ . In these plots, all trajectories start from the origin, and the dotted diagonal line mimics a bad unlearning algorithm: tracing the top-left of the dotted diagonal line (e.g., GA, KL for $\rho = 0.75$) indicates that model performance deteriorates on both the forget and retain data, resulting in collateral forgetting. In contrast, tracing the bottom-right of the dotted diagonal

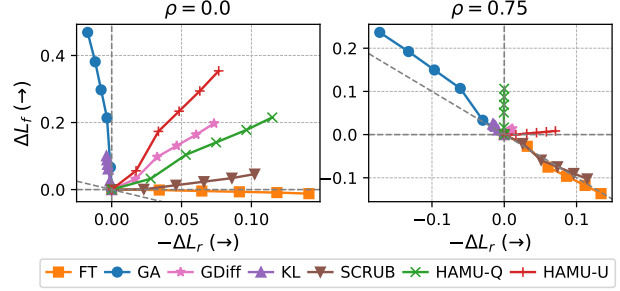


Figure 4. Comparison of HAMU with baselines for unlearning scenarios of different levels of hardness. HAMU outperforms baselines in both easy ($\rho = 0$) and hard ($\rho = 0.75$) scenarios. When unlearning becomes harder (e.g., $\rho = 0.75$), most baseline methods fail to satisfy both objectives.

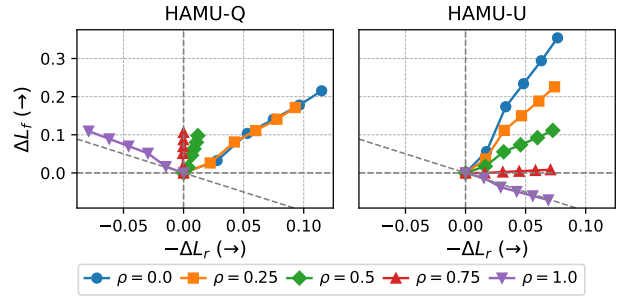


Figure 5. HAMU-Q enforces a positive ΔL_f while HAMU-U enforces a positive $-\Delta L_r$.

line (e.g., FT, SCRUB for $\rho = 0.75$) indicates that model performance improves on both the forget and retain data, suggesting poor forget quality. An ideal unlearning algorithm should achieve positive ΔL_f and $-\Delta L_r$, i.e., its unlearning trajectory should trend toward the top-right region. In the easy scenario ($\rho = 0$ with dissimilar forget and retain data), GA and KL achieve a large ΔL_f with a negative $-\Delta L_r$, FT and SCRUB achieve a large $-\Delta L_r$ with a negative ΔL_f , while HAMU variants and GDiff desirably achieve large positive ΔL_f and $-\Delta L_r$, improving both objectives simultaneously. However, in the hard scenario ($\rho = 0.75$ with similar forget and retain data), only HAMU variants achieve non-negligible positive ΔL_f or $-\Delta L_r$ without hurting the other objective. Additional experimental results for different levels of similarity mixing ratio ρ are in App. H.1.

HAMU enforces the desired objectives. Fig. 5 compares the unlearning trajectories of HAMU-Q and HAMU-U. Across all ρ ’s and epochs, we observe that HAMU-Q indeed enforces a positive ΔL_f (better forget quality), and HAMU-U enforces a positive $-\Delta L_r$ (better retain utility). This is consistent with our formulation in Problem (1) and Problem (16), respectively. Fig. 5 shows that as the forget data becomes more similar to the retain data (higher

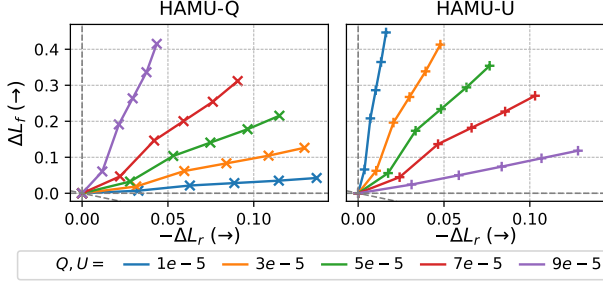


Figure 6. Unlearning trajectories of HAMU-Q and HAMU-U for different values of Q and U when $\rho = 0$.

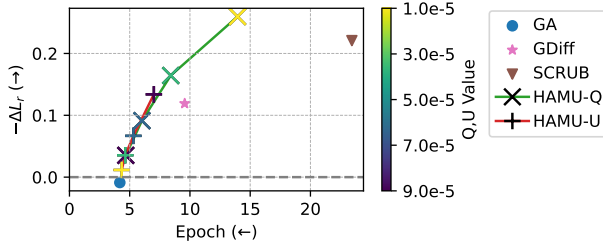


Figure 7. Plot of improvement in retain utility $-\Delta L_r$ against number of epochs required to reach a specified threshold for L_f . Methods that failed to reach this threshold have been excluded.

ρ), $-\Delta L_r$ decreases for HAMU-Q and ΔL_f decreases for HAMU-U. When $\rho = 1$, unlearning becomes so hard that even HAMU-Q results in collateral forgetting and HAMU-U fails to unlearn the forget data. Note that the stopping criterion was not enforced in these experiments to illustrate the results for these hard unlearning scenarios.

Selection of constraints Q and U trades off between improvement in forget quality and retain utility. We illustrate the trade-off between ΔL_f and $-\Delta L_r$ for different values of Q and U in Fig. 6. When using HAMU-Q, increasing Q results in a better forget quality (higher ΔL_f), at the expense of a worse retain utility (lower $-\Delta L_r$). Conversely, when using HAMU-U, increasing U results in a better retain utility, at the expense of a worse forget quality. In practice, we leave the selection of Q and U to the user when using HAMU based on their specific requirements.

Selection of constraints Q and U trades off between number of epochs and objective improvement. In Fig. 7, we plot $-\Delta L_r$ against the number of unlearning epochs required to reach a pre-specified target forget loss $L_f = 1.0$. We observe that higher Q requires fewer epochs (i.e., lower computation cost) to reach the required forget quality, but leads to worse retain utility. If more epochs can be afforded, the user can choose a smaller Q , in order to achieve a better retain utility. Fig. 7 also shows that the Pareto-front of HAMU outperforms the baselines, with the Pareto-front of HAMU-Q overlapping with that of HAMU-U.

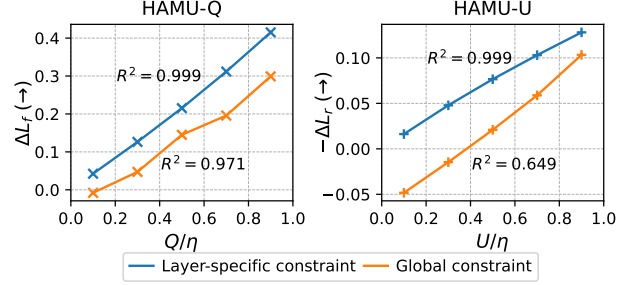


Figure 8. Plots of ΔL_f vs. Q/η and $-\Delta L_r$ vs. U/η under layer-wise and global constraint for HAMU-Q and HAMU-U, respectively. We assume linear proportionality through origin for R^2 .

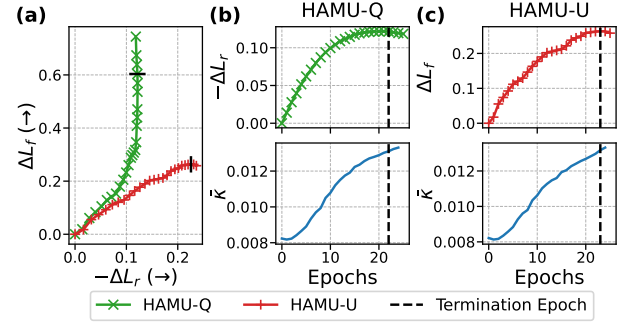


Figure 9. Unlearning trajectories for 25 epochs at $\rho = 0.5$, along with the estimated hardness $\bar{\kappa}$ at each epoch. Termination epoch refers to the epoch where the stopping criterion is met.

Layer-wise update outperforms global update in unlearning performance. In addition to the practical benefits discussed in Sec. 5.1, Fig. 8 shows that using layer-specific constraints in HAMU weight updates achieves better forget quality and retain utility than using global constraint in the updates for HAMU-Q and HAMU-U, respectively. Fig. 8 also shows that ΔL_f and $-\Delta L_r$ are undesirably negative for small Q and U under global constraint. In addition, we observe an almost perfect linear proportional relationship ($R^2 = 0.999$) between the user-specified requirements, Q or U , and the corresponding ΔL_f or $-\Delta L_r$ for HAMU-Q and HAMU-U, respectively. This may be because the unlearning update is computed layer-wise, which allows a more accurate linear approximation of each layer’s contribution to the overall loss, whereas the global update approach combines gradients across different layers and introduces additional nonlinearities.

Hardness increases as unlearning progresses. The bottom plots of Fig. 9b-c show that unlearning hardness increases with unlearning epochs. This may be because as unlearning progresses, there is less room for maneuvering, and fewer update directions remain feasible for simultaneously improving both objectives. The increase in hardness leads to a decrease in the rate of change for $-\Delta L_r$ and

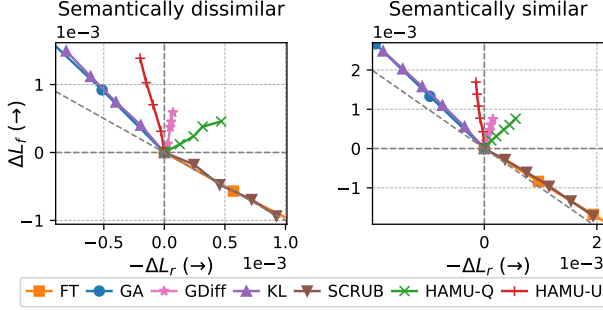


Figure 10. Comparison of HAMU with baselines for different levels of semantic similarity on LLM QA task. The plot is zoomed in, with parts of FT and GA (which perform poorly) cropped out.

ΔL_f for HAMU-Q and HAMU-U, respectively, as shown in the top plots of Fig. 9b-c.

Stopping criterion terminates HAMU when collateral forgetting is inevitable. Fig. 9 b-c shows that once $-\Delta L_r$ and ΔL_f start to drop for HAMU-Q and HAMU-U, respectively, our stopping criterion terminates unlearning. Note that this can happen at an epoch slightly after the turning point, as shown in the top plot of Fig. 9b, likely due to the linear approximation errors in HAMU. In practice, a model owner performing unlearning can avoid this by terminating unlearning when the stopping criterion is close to being met, i.e., when $\bar{\kappa} > \bar{\kappa}_2 - \varepsilon$ for some small $\varepsilon > 0$. In App. C, we discuss how to adapt the stopping criterion for layer-specific constraints.

6.2. LLM Question Answering Task

We additionally evaluate HAMU on LLM *question answering* (QA) task, a more recent and larger scale unlearning setup. We use the Llama-2-7b-chat-hf model⁵ trained on the unlearning WaterDrum-TOFU dataset (Lu et al., 2026) which has subsets with different levels of similarity. For our experiments, we use the “No duplicate” and “Semantic duplicate” subsets of WaterDrum-TOFU, where the latter includes paraphrased versions of the forget data in the retain data. We refer to the settings as “Semantically dissimilar” and “Semantically similar”, respectively. Details on the training process are in App. F.3. We also include results for additional baselines and evaluation metrics such as model utility and *membership inference attack* (MIA) success rate in App. H.2.

Unlearning is harder when the forget and retain data are more semantically similar. The average estimated hardness for the “semantically similar” setting ($\bar{\kappa} = 6.1e-4$) is higher than that of the “semantically dissimilar” setting ($\bar{\kappa} = 4.0e-4$), consistent with our intuition that it

⁵<https://huggingface.co/meta-llama/Llama-2-7b-chat-hf>

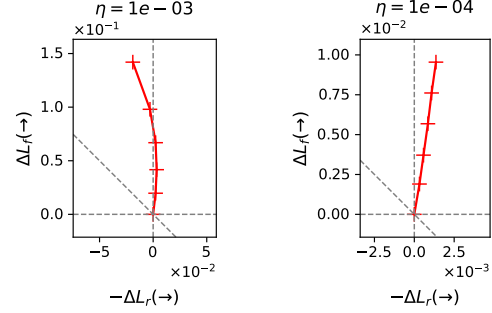


Figure 11. Comparison of HAMU-U unlearning trajectories for different learning rates η , with the same $U = 0.6\eta$, under the semantically dissimilar LLM experimental setup.

is easier to unlearn the forget data that is less similar to the retain data. Fig. 10 shows that HAMU-Q is able to improve both forget quality and retain utility for both settings.

LLM unlearning is hard. In Fig. 10, only HAMU-Q and GDiff can improve both forget quality and retain utility simultaneously. Comparatively, HAMU-Q is able to achieve a greater increase in forget quality and retain utility than GDiff. Most baselines perform poorly, tracing close to the dotted diagonal which mimics a bad unlearning algorithm, even in the easier setting with semantically dissimilar forget and retain data. Note that HAMU-U achieves a slightly negative $-\Delta L_r$ in Fig. 10, due to poor linear approximation, i.e., the positive retain utility improvement constraint is satisfied by the approximated problem but not the original problem. Fig. 11 shows that with a smaller learning rate, the retain objective improves from violating to satisfying the constraint due to better approximation.

7. Conclusion

We introduce HAMU, a novel hardness-aware multi-objective unlearning algorithm that adjusts updates based on per-iteration hardness and terminates when collateral forgetting is reached. Through theoretical analysis of a constrained optimization problem, HAMU optimally reduces the cost of forgetting while satisfying the forget quality constraint. Naturally arising from the analysis, we propose using the gradient dot product as the hardness measure to quantify the difficulty of reconciling the unlearning objectives. This proposed measure theoretically predicts the unlearning performance, formalizing the intuition that more similar data are harder to unlearn. Our experiments demonstrate HAMU’s superior performance compared to existing baselines, and its applicability and scalability to large, non-convex models. We believe HAMU represents a significant step toward developing unlearning algorithms that are both practical and theoretically-grounded, enabling broader adoption of unlearning in real-world scenarios. We discuss the relevant literature in App. A.

Acknowledgements

This research is supported by the National Research Foundation, Singapore under its AI Singapore Programme (AISG Award No: AISG3-RP-2022-029). This research/project is supported by the National Research Foundation, Singapore under its National Large Language Models Funding Initiative (AISG Award No: AISG-NMLP-2024-001). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore. Jiangwei Chen is supported by the Institute for Infocomm Research (I²R), Agency for Science, Technology and Research (A*STAR). Xinyuan Niu is supported by the Centre for Frontier AI Research (CFAR), Agency for Science, Technology and Research, Singapore (A*STAR). We would like to thank the anonymous reviewers and the AC for their helpful and constructive feedback.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Allouah, Y., Kazdan, J., Guerraoui, R., and Koyejo, S. The utility and complexity of in- and out-of-distribution machine unlearning. In *Proc. ICLR*, 2025.
- Block, J. L., Mokhtari, A., and Shakkottai, S. Machine unlearning under overparameterization. In *Proc. ICML Workshop on Machine Unlearning for Generative AI*, 2025.
- Bourtole, L., Chandrasekaran, V., Choquette-Choo, C. A., Jia, H., Travers, A., Zhang, B., Lie, D., and Papernot, N. Machine unlearning. In *Proc. IEEE S&P*, 2021.
- Bui, N., Lu, X., Sim, R. H. L., Ng, S.-K., and Low, B. K. H. How to cure Newton for unlearning neural networks? an empirical study from the Hessian perspective. In *Proc. ICLR*, 2026.
- Cao, Y. and Yang, J. Towards making systems forget with machine unlearning. In *Proc. IEEE S&P*, 2015.
- Chen, J. and Yang, D. Unlearn what you want to forget: Efficient unlearning for LLMs. In *Proc. EMNLP*, 2023.
- Choromanska, A., Henaff, M., Mathieu, M., Arous, G. B., and LeCun, Y. The loss surfaces of multilayer networks. In *Proc. AISTATS*, 2015.
- Duan, H., Yang, Y., Abbasi, A., and Tam, K. Y. Ready2Unlearn: A learning-time approach for preparing models with future unlearning readiness. arXiv:2505.10845, 2025.
- Eldan, R. and Russinovich, M. Who’s Harry Potter? approximate unlearning in LLMs. arXiv:2310.02238, 2023.
- Fan, C., Liu, J., Lin, L., Jia, J., Zhang, R., Mei, S., and Liu, S. Simplicity prevails: Rethinking negative preference optimization for LLM unlearning. In *Proc. NeurIPS*, 2025.
- Georgiev, K., Rinberg, R., Park, S. M., Garg, S., Ilyas, A., Madry, A., and Neel, S. Attribute-to-delete: Machine unlearning via datamodel matching. In *Proc. ICLR*, 2025.
- Gugger, S., Debut, L., Wolf, T., Schmid, P., Mueller, Z., Mangrulkar, S., Sun, M., and Bossan, B. Accelerate: Training and inference at scale made simple, efficient and adaptable. <https://github.com/huggingface/accelerate>, 2022.
- Guo, C., Goldstein, T., Hannun, A., and Van Der Maaten, L. Certified data removal from machine learning models. In *Proc. ICML*, 2020.
- Haimes, Y. Y., Lasdon, L. S., and Wismer, D. A. On a bi-criterion formulation of the problems of integrated system identification and system optimization. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-1(3): 296–297, 1971.
- Halimi, A., Kadhe, S., Rawat, A., and Baracaldo, N. Federated unlearning: How to efficiently erase a client in FL? In *Proc. ICML Workshop on Updatable Machine Learning*, 2022.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proc. CVPR*, 2016.
- Hu, E. J., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. LoRA: Low-rank adaptation of large language models. In *Proc. ICLR*, 2022.
- Jang, J., Yoon, D., Yang, S., Cha, S., Lee, M., Logeswaran, L., and Seo, M. Knowledge unlearning for mitigating privacy risks in language models. In *Proc. ACL*, 2023.
- Jin, X., Bu, Z., Vinzamuri, B., Ramakrishna, A., Chang, K.-W., Cevher, V., and Hong, M. Unlearning as multi-task optimization: A normalized gradient difference approach with an adaptive learning rate. In *Proc. NAACL*, 2025.

- Khan, W., Leem, S., See, K. B., Wong, J. K., Zhang, S., and Fang, R. A comprehensive survey of foundation models in medicine. *IEEE Reviews in Biomedical Engineering*, pp. 1–22, 2025.
- Krizhevsky, A. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. URL <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.
- Kurmanji, M., Triantafillou, P., Hayes, J., and Triantafillou, E. Towards unbounded machine unlearning. In *Proc. NeurIPS*, 2023.
- Lau, G. K. R., Niu, X., Dao, H., Chen, J., Foo, C.-S., and Low, B. K. H. Waterfall: Scalable framework for robust text watermarking and provenance for LLMs. In *Proc. EMNLP*, 2024.
- Lee, J. D., Simchowitz, M., Jordan, M. I., and Recht, B. Gradient descent only converges to minimizers. In *Proc. COLT*, 2016.
- Lin, S., Zhang, X., Susilo, W., Chen, X., and Liu, J. GDR-GMA: Machine unlearning via direction-rectified and magnitude-adjusted gradients. In *Proc. ACM MM*, 2024.
- Liu, B., Liu, X., Jin, X., Stone, P., and Liu, Q. Conflict-averse gradient descent for multi-task learning. In *Proc. NeurIPS*, 2021.
- Liu, Y., Fan, M., Chen, C., Liu, X., Ma, Z., Wang, L., and Ma, J. Backdoor defense with machine unlearning. In *Proc. IEEE INFOCOM*, 2022.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *Proc. ICLR*, 2019.
- Lu, X., Niu, X., Lau, G. K. R., Nhung, B. T. C., Sim, R. H. L., Wen, F., Foo, C.-S., Ng, S.-K., and Low, B. K. H. Waterdrum: Watermarking for data-centric unlearning metric. In *Proc. ICLR*, 2026.
- Maini, P., Feng, Z., Schwarzschild, A., Lipton, Z. C., and Kolter, J. Z. TOFU: A task of fictitious unlearning for LLMs. In *Proc. COLM*, 2024.
- Mavrothalassitis, I., Puigdemont, P., Levi, N. I., and Cevher, V. Ascent fails to forget. In *Proc. NeurIPS*, 2025.
- Neel, S., Roth, A., and Sharifi-Malvajerdi, S. Descent-to-delete: Gradient-based methods for machine unlearning. In *Proc. ALT*, 2021.
- Nguyen, Q. P., Low, B. K. H., and Jaillet, P. Variational bayesian unlearning. In *Proc. NeurIPS*, 2020.
- Nguyen, Q. P., Oikawa, R., Divakaran, D. M., Chan, M. C., and Low, B. K. H. Markov chain Monte Carlo-based machine unlearning: Unlearning what needs to be forgotten. In *Proc. AsiaCCS*, 2022.
- Pawelczyk, M., Neel, S., and Lakkaraju, H. In-context unlearning: Language models as few-shot unlearners. In *Proc. ICML*, 2024.
- Raghu, M., Gilmer, J., Yosinski, J., and Sohl-Dickstein, J. SVCCA: Singular vector canonical correlation analysis for deep learning dynamics and interpretability. In *Proc. NeurIPS*, 2017.
- Raza, M., Jahangir, Z., Riaz, M. B., Saeed, M. J., and Sattar, M. A. Industrial applications of large language models. *Scientific Reports*, 15(1):13755, Apr 2025.
- Santurkar, S., Tsipras, D., Ilyas, A., and Madry, A. How does batch normalization help optimization? In *Proc. NeurIPS*, 2018.
- Sarvmaili, M., Sajjad, H., and Wu, G. Towards understanding the feasibility of machine unlearning. arXiv:2410.03043, 2024.
- Shi, W., Ajith, A., Xia, M., Huang, Y., Liu, D., Blevins, T., Chen, D., and Zettlemoyer, L. Detecting pretraining data from large language models. In *Proc. ICLR*, 2024.
- Skean, O., Arefin, M. R., Zhao, D., Patel, N. N., Naghiyev, J., Lecun, Y., and Schwartz-Ziv, R. Layer by layer: Uncovering hidden representations in language models. In *Proc. ICML*, 2025.
- Tian, B., Liang, X., Cheng, S., Liu, Q., Wang, M., Sui, D., Chen, X., Chen, H., and Zhang, N. To forget or not? Towards practical knowledge unlearning for large language models. In *Findings of EMNLP*, 2024.
- Ullah, E., Mai, T., Rao, A., Rossi, R. A., and Arora, R. Machine unlearning via algorithmic stability. In *Proc. COLT*, 2021.
- Wang, Q., Zhou, J. P., Zhou, Z., Shin, S., Han, B., and Weinberger, K. Q. Rethinking LLM unlearning objectives: A gradient perspective and go beyond. In *Proc. ICLR*, 2025a.
- Wang, Y., Wang, Q., Liu, F., Huang, W., Du, Y., Du, X., and Han, B. GRU: Mitigating the trade-off between unlearning and retention for LLMs. In *Proc. ICML*, 2025b.
- Wu, Y., Dobriban, E., and Davidson, S. DeltaGrad: Rapid retraining of machine learning models. In *Proc. ICML*, 2020.

- Wu, Y., Zhou, S., Yang, M., Wang, L., Chang, H., Zhu, W., Hu, X., Zhou, X., and Yang, X. Unlearning concepts in diffusion model via concept domain correction and concept preserving gradient. In *Proc. AAAI*, 2025.
- Yu, T., Kumar, S., Gupta, A., Levine, S., Hausman, K., and Finn, C. Gradient surgery for multi-task learning. In *Proc. NeurIPS*, 2020.
- Zhang, R., Lin, L., Bai, Y., and Mei, S. Negative preference optimization: From catastrophic collapse to effective unlearning. In *Proc. COLM*, 2024.
- Zhao, K., Kurmanji, M., Bărbulescu, G.-O., Triantafillou, E., and Triantafillou, P. What makes unlearning hard and what to do about it. In *Proc. NeurIPS*, 2024.

A. Related Work

One common objective of machine unlearning is to approximate the retrained model (Georgiev et al., 2025; Nguyen et al., 2022), which is equivalent to minimizing the retain loss (Neel et al., 2021; Bourtole et al., 2021). Another prevalent objective is to increase the forget loss (Halimi et al., 2022; Zhang et al., 2024). As neither of these objectives alone suffices to ensure a model with both strong forget and retain performance (Wang et al., 2025a; Fan et al., 2025), multi-objective unlearning (Jin et al., 2025) aims to optimize both objectives simultaneously.

One straightforward approach to multi-objective unlearning is to directly optimize a weighted combination of the loss functions (Kurmanji et al., 2023; Duan et al., 2025). Other approaches such as GDR-GMA (Lin et al., 2024) and GRU (Wang et al., 2025b) combine the retain gradient with the forget gradient to compute the unlearning update. However, neither approach guarantees that a certain objective is achieved, nor do they adjust the update based on the hardness to optimally reduce the cost of forgetting.

Reconciling the retain and forget objectives in machine unlearning can be viewed as a specific instance of multi-objective optimization, and therefore multi-task learning approaches such as PCGrad (Yu et al., 2020) and CAGrad (Liu et al., 2021) can be adapted for multi-objective unlearning. Our approach is closely related to the classical ε -constraint method (Haimes et al., 1971) but differs from standard approaches in that, instead of running gradient descent on the global constrained optimization problem, we derive a per-iteration, first-order update rule that enforces a minimum improvement in forget quality while minimizing retain utility degradation. The novelty lies in this local, gradient-level formulation (Sec. 4), as well as in the hardness characterization and its interpretation, which directly inform the design of our algorithm (Sec. 5) and enable efficient application to large-scale, non-convex models.

Unlearning methods with theoretical guarantees (Guo et al., 2020; Ullah et al., 2021) typically require strong assumptions, such as strongly convex loss functions and the existence of a unique minimizer (Wu et al., 2020; Allouah et al., 2025), limiting their practicality. Existing works relevant to unlearning hardness (Zhao et al., 2024; Sarvmaili et al., 2024) are largely heuristic-based and do not establish a formal connection between the proposed hardness measures and unlearning performance.

B. Proofs and Derivations

B.1. First-Order Loss Approximation

For any $\mathbf{z}_r = (\mathbf{x}_r, y_r) \in D_r$, applying Taylor expansion to $L_{t+1}(\mathbf{z}_r)$, we have

$$L_{t+1}(\mathbf{z}_r) = L_t(\mathbf{z}_r) + \Delta \mathbf{w}^\top \nabla L_t(\mathbf{z}_r) + \underbrace{\frac{1}{2} \Delta \mathbf{w}^\top \mathbf{H} \Delta \mathbf{w}}_E,$$

where E is the approximation error term, and $\mathbf{H} = \nabla^2 L(\mathbf{p}(\mathbf{x}_r; \mathbf{w}'), y_r)$ is the Hessian at some intermediate model $\mathbf{w}' = \mathbf{w}_t + c\Delta \mathbf{w}$ for some scalar $c \in (0, 1)$. As the model update $\Delta \mathbf{w}$ is induced by a single gradient descent step on $\mathbf{z}_r$, substituting $\Delta \mathbf{w} = -\eta \nabla L_t(\mathbf{z}_r)$ gives

$$\Delta_{\mathbf{z}_r} L_t(\mathbf{z}_r) = L_{t+1}(\mathbf{z}_r) - L_t(\mathbf{z}_r) = -\eta \|\nabla L_t(\mathbf{z}_r)\|^2 + E.$$

The approximation quality can be measured by the ratio between the absolute values of the error term and linear term:

$$\frac{|E|}{|\Delta \mathbf{w}^\top \nabla L_t(\mathbf{z}_r)|} = \left| \frac{\frac{1}{2} \Delta \mathbf{w}^\top \mathbf{H} \Delta \mathbf{w}}{-\eta \|\nabla L_t(\mathbf{z}_r)\|^2} \right| \leq \left| \frac{\frac{1}{2} \eta^2 \|\nabla L_t(\mathbf{z}_r)\|^2 \lambda_{\max}}{-\eta \|\nabla L_t(\mathbf{z}_r)\|^2} \right| = \frac{1}{2} \eta \lambda_{\max},$$

where $\lambda_{\max}$ is the maximum eigenvalue of $\mathbf{H}$. Empirical evidence (Santurkar et al., 2018; Bui et al., 2026) has shown that the eigenvalues of Hessian typically falls below 200 for neural networks on standard datasets such as CIFAR-10. Combined with a small learning rate of $1e-4$ used in our experiments (Sec. 6), the error ratio falls below 1%. Therefore, the second-order derivative component is negligible in practice and using the first-order derivative component is sufficient in each iteration. Furthermore, we are redoing the approximation at each iteration, thereby ensuring consistent approximation along the unlearning trajectory. Since our formulation in Problem (1) relies on approximation, the derived algorithms can be applied to any non-convex model as long as a sufficiently small learning rate is chosen to ensure approximation quality, the theoretical results also only hold up to the extent of approximation. Despite this, our methods still perform well on non-convex models as demonstrated in our experiments (Sec. 6).

B.2. The Forget-Constrained Problem

Theorem 4.1 and **Proposition 4.3** are proven below by solving a convex optimization problem using KKT conditions.

As Problem (1) enforces a constraint on the forget quality, we refer to it as the forget-constrained problem. Alternatively, we can enforce a constraint on the retain utility, we discuss this alternative retain-constrained problem in App. B.6. To solve Problem (1), first note that it is feasible when $Q \leq R \|g_f\|$. We assume this feasibility condition holds henceforth. The Lagrangian of Problem (1) is given by

$$\mathcal{L}(\Delta w, \alpha, \beta) = g_r \cdot \Delta w + \alpha(Q - g_f \cdot \Delta w) + \beta(\Delta w \cdot \Delta w - R^2).$$

Since Problem (1) is a convex optimization problem with convex inequality constraints, we can find its optimal value by solving the KKT conditions:

Stationarity Condition	$\frac{\partial \mathcal{L}}{\partial \Delta w} = g_r - \alpha g_f + 2\beta \Delta w = 0$
Complementary Slackness	$\alpha(Q - g_f \cdot \Delta w) = 0$ $\beta(\Delta w \cdot \Delta w - R^2) = 0$
Primal Feasibility	$Q - g_f \cdot \Delta w \leq 0$ $\Delta w \cdot \Delta w - R^2 \leq 0$
Dual Feasibility	$\alpha \geq 0, \beta \geq 0.$

We discuss all possible cases corresponding to different values of the dual variables α and β . Note that when the dual variables are positive, the corresponding primal feasibility conditions must bind. Specifically, when $\alpha > 0$, $g_f \cdot \Delta w = Q$; when $\beta > 0$, $\|\Delta w\| = R$.

Case 1: $\alpha = 0, \beta > 0$.

When $\alpha = 0$, the stationarity condition simplifies to

$$g_r + 2\beta \Delta w = 0,$$

which implies

$$\Delta w = -\frac{1}{2\beta} \cdot g_r. \tag{5}$$

To find the value of β , we equate the norm of the above expression to R since $\|\Delta w\| = R$ when $\beta > 0$:

$$\|\Delta w\|^2 = \frac{1}{4\beta^2} \|g_r\|^2 = R^2.$$

Solving for β , we obtain

$$\beta = \frac{\|g_r\|}{2R}.$$

Substitute the value of β into Eq. (5), we obtain the direct update

$$\Delta w^* = -\frac{R}{\|g_r\|} \cdot g_r,$$

which gives the optimal value

$$F_r^* = g_r \cdot \Delta w^* = -R \|g_r\|.$$

To show that the direct update corresponds to the case when $\kappa \leq \kappa_1$, we check that Δw^* should satisfy primal feasibility:

$$g_f \cdot \Delta w^* \geq Q \implies g_r \cdot g_f \leq -\frac{Q \|g_r\|}{R},$$

which is exactly $\kappa \leq \kappa_1$.

Case 2: $\alpha > 0, \beta > 0$.

We first decompose $\mathbf{g}_r$ into components that are parallel and orthogonal to $\mathbf{g}_f$. Let $\hat{\mathbf{g}}_f \triangleq \mathbf{g}_f / \|\mathbf{g}_f\|$ be the unit vector of $\mathbf{g}_f$, then $\mathbf{g}_r = x \cdot \hat{\mathbf{g}}_f + \mathbf{g}_{r\perp}$, where $x = \mathbf{g}_r \cdot \hat{\mathbf{g}}_f$ is the magnitude of the parallel component, and $\mathbf{g}_{r\perp} \perp \mathbf{g}_f$ is the orthogonal component. Similarly we can decompose $\Delta \mathbf{w}$ into $\Delta \mathbf{w} = y \cdot \hat{\mathbf{g}}_f + \Delta \mathbf{w}_\perp$, where $\Delta \mathbf{w}_\perp \perp \mathbf{g}_f$. When $\alpha > 0$, $\mathbf{g}_f \cdot \Delta \mathbf{w} = Q$, substitute the decomposed $\Delta \mathbf{w}$ we obtain

$$\mathbf{g}_f \cdot (y \cdot \hat{\mathbf{g}}_f + \Delta \mathbf{w}_\perp) = Q,$$

which gives the magnitude of $\Delta \mathbf{w}$'s parallel component to $\mathbf{g}_f$,

$$y = \frac{Q}{\|\mathbf{g}_f\|}.$$

By Pythagorean theorem, the magnitude of $\Delta \mathbf{w}$'s orthogonal component is

$$\|\Delta \mathbf{w}_\perp\|^2 = \|\Delta \mathbf{w}\|^2 - y^2 = R^2 - \frac{Q^2}{\|\mathbf{g}_f\|^2}.$$

From the stationarity condition we know $\Delta \mathbf{w} \in \text{span}\{\mathbf{g}_r, \mathbf{g}_f\}$, thus $\Delta \mathbf{w}_\perp$ is parallel to $\mathbf{g}_{r\perp}$. As we want to minimize $\mathbf{g}_r \cdot \Delta \mathbf{w}$, we will choose $\Delta \mathbf{w}_\perp$ in the opposite direction of $\mathbf{g}_{r\perp}$ so that $\Delta \mathbf{w}_\perp \cdot \mathbf{g}_{r\perp} < 0$. Hence, the rectified update is

$$\Delta \mathbf{w}^* = \frac{Q}{\|\mathbf{g}_f\|^2} \cdot \mathbf{g}_f - \sqrt{R^2 - \frac{Q^2}{\|\mathbf{g}_f\|^2}} \cdot \frac{\mathbf{g}_{r\perp}}{\|\mathbf{g}_{r\perp}\|}, \quad (6)$$

where $\mathbf{g}_{r\perp} = \mathbf{g}_r - x \cdot \hat{\mathbf{g}}_f = \mathbf{g}_r - \frac{\mathbf{g}_r \cdot \mathbf{g}_f}{\|\mathbf{g}_f\|^2} \cdot \mathbf{g}_f$. Thus, the optimal value F_r^* is

$$\begin{aligned} \mathbf{g}_r \cdot \Delta \mathbf{w}^* &= \frac{Q}{\|\mathbf{g}_f\|^2} (\mathbf{g}_r \cdot \mathbf{g}_f) - \|\mathbf{g}_{r\perp}\| \sqrt{R^2 - \frac{Q^2}{\|\mathbf{g}_f\|^2}} \\ &= \frac{Q}{\|\mathbf{g}_f\|^2} (\mathbf{g}_r \cdot \mathbf{g}_f) - \sqrt{\left(\|\mathbf{g}_r\|^2 - \frac{(\mathbf{g}_r \cdot \mathbf{g}_f)^2}{\|\mathbf{g}_f\|^2}\right) \left(R^2 - \frac{Q^2}{\|\mathbf{g}_f\|^2}\right)}. \end{aligned} \quad (7)$$

Next, we will show how **Case 2 corresponds precisely to the scenario in which the boundary condition fails**, i.e., the rectified update is applied when $\kappa > \kappa_1$, thus **justifying our use of hardness measure to inform the unlearning update**.

By decomposing the stationarity condition in the direction parallel to $\mathbf{g}_f$ and the direction orthogonal to $\mathbf{g}_f$,

$$x - \alpha \|\mathbf{g}_f\| + 2\beta y = 0 \quad \text{and} \quad \mathbf{g}_{r\perp} + 2\beta \Delta \mathbf{w}_\perp = \mathbf{0},$$

we obtain the expressions for the dual variables:

$$\alpha = \frac{x + 2\beta y}{\|\mathbf{g}_f\|} \quad \text{and} \quad \beta = \frac{\|\mathbf{g}_{r\perp}\|}{2\|\Delta \mathbf{w}_\perp\|}.$$

From $\alpha > 0$, and substituting the value of β , we have

$$\begin{aligned} x + 2\beta y &> 0 \\ x + 2 \cdot \frac{\|\mathbf{g}_{r\perp}\|}{2\|\Delta \mathbf{w}_\perp\|} \cdot y &> 0 \\ x \|\Delta \mathbf{w}_\perp\| + y \|\mathbf{g}_{r\perp}\| &> 0. \end{aligned} \quad (8)$$

If $\mathbf{g}_r \cdot \mathbf{g}_f > 0$, then clearly $\kappa = \mathbf{g}_r \cdot \mathbf{g}_f > -Q \|\mathbf{g}_r\| / R = \kappa_1$ and we are done.

If $\mathbf{g}_r \cdot \mathbf{g}_f < 0$, substituting the values of x, y and norms $\|\Delta \mathbf{w}_\perp\|, \|\mathbf{g}_{r\perp}\|$ into Ineq. (8),

$$\begin{aligned}
 y \|\mathbf{g}_{r\perp}\| &> -x \|\Delta \mathbf{w}_\perp\| \\
 \frac{Q}{\|\mathbf{g}_f\|} \sqrt{\|\mathbf{g}_r\|^2 - \frac{(\mathbf{g}_r \cdot \mathbf{g}_f)^2}{\|\mathbf{g}_f\|^2}} &> -\frac{\mathbf{g}_r \cdot \mathbf{g}_f}{\|\mathbf{g}_f\|} \sqrt{R^2 - \frac{Q^2}{\|\mathbf{g}_f\|^2}} \\
 \frac{Q^2}{\|\mathbf{g}_f\|^2} \left(\|\mathbf{g}_r\|^2 - \frac{(\mathbf{g}_r \cdot \mathbf{g}_f)^2}{\|\mathbf{g}_f\|^2} \right) &> \frac{(\mathbf{g}_r \cdot \mathbf{g}_f)^2}{\|\mathbf{g}_f\|^2} \left(R^2 - \frac{Q^2}{\|\mathbf{g}_f\|^2} \right) \\
 Q^2 \|\mathbf{g}_r\|^2 &> R^2 (\mathbf{g}_r \cdot \mathbf{g}_f)^2 \\
 Q \|\mathbf{g}_r\| &> -R (\mathbf{g}_r \cdot \mathbf{g}_f) \\
 \mathbf{g}_r \cdot \mathbf{g}_f &> -\frac{Q \|\mathbf{g}_r\|}{R}.
 \end{aligned}$$

Thus, Case 2 is precisely defined by $\mathbf{g}_r \cdot \mathbf{g}_f > -Q \|\mathbf{g}_r\| / R$, which is exactly $\kappa > \kappa_1$.

For Cases 3 and 4, we first solve for the optimal values and updates, then show they are degenerate cases of Case 2.

Case 3: $\alpha = 0, \beta = 0$.

When $\beta = 0$, the stationarity condition implies $\mathbf{g}_r = \alpha \cdot \mathbf{g}_f$. Further, $\alpha = 0$ implies $\mathbf{g}_r = \mathbf{0}$ so $\mathbf{g}_r \cdot \Delta \mathbf{w} = 0$ for any feasible $\Delta \mathbf{w}$. Hence, we will choose the minimum-norm solution that satisfies primal feasibility, i.e., $\Delta \mathbf{w}^* = (Q / \|\mathbf{g}_f\|^2) \cdot \mathbf{g}_f$. The optimum F_r^* is 0 since $\mathbf{g}_r = \mathbf{0}$.

Case 4: $\alpha > 0, \beta = 0$.

From $\alpha > 0$, the primal feasibility must bind: $\mathbf{g}_f \cdot \Delta \mathbf{w} = Q$, which puts $\Delta \mathbf{w}$ at the boundary of the primal feasibility condition. We will again choose the minimum-norm solution $\Delta \mathbf{w}^* = (Q / \|\mathbf{g}_f\|^2) \cdot \mathbf{g}_f$, then $\mathbf{g}_r \cdot \Delta \mathbf{w}^* = \kappa Q / \|\mathbf{g}_f\|^2$.

To show that the Cases 3 and 4 are subsumed by Case 2 ($\kappa > \kappa_1$), we need to show

- $\mathbf{g}_r \cdot \mathbf{g}_f > -Q \|\mathbf{g}_r\| / R$ is satisfied under Cases 3 and 4;
- Consistency with Eq. (6) for $\Delta \mathbf{w}^*$ under Cases 3 and 4;
- Consistency with Eq. (7) for $\mathbf{g}_r \cdot \Delta \mathbf{w}^*$ under Cases 3 and 4.

In Case 3, $\mathbf{g}_r \cdot \mathbf{g}_f = 0 > -Q \|\mathbf{g}_r\| / R$. Also, substituting $\mathbf{g}_r = \mathbf{0}$ into the expression for $\mathbf{g}_{r\perp}$ leads to $\mathbf{g}_{r\perp} = \mathbf{0}$, so $\Delta \mathbf{w}^*$ in Case 2 degenerates to $(Q / \|\mathbf{g}_f\|^2) \cdot \mathbf{g}_f$. The optimal value in Case 2 also evaluates to 0 when $\mathbf{g}_r = \mathbf{0}$.

In Case 4, from $\mathbf{g}_r = \alpha \cdot \mathbf{g}_f$ we have $\alpha = (\mathbf{g}_r \cdot \mathbf{g}_f) / \|\mathbf{g}_f\|^2$. Then $\alpha > 0$ implies $\mathbf{g}_r \cdot \mathbf{g}_f > 0 > -Q \|\mathbf{g}_r\| / R$. The optimal update and value evaluate to $(Q / \|\mathbf{g}_f\|^2) \cdot \mathbf{g}_f$ and $(\mathbf{g}_r \cdot \mathbf{g}_f) Q / \|\mathbf{g}_f\|^2$, respectively, since $\mathbf{g}_{r\perp} = \mathbf{0}$ given the collinearity between $\mathbf{g}_r$ and $\mathbf{g}_f$.

Therefore, **our update rule and optimal value encompass all possible cases of Problem (1).**

B.3. Relationship Between Unlearning Performance and the Hardness Measure

The proof of Corollary 4.2 is given below.

Proof. When $\kappa \leq \kappa_1$, the function is constant. When $\kappa > \kappa_1$, taking the derivative of F_r^* with respect to κ yields

$$\frac{dF_r^*}{d\kappa} = \frac{Q}{\|\mathbf{g}_f\|^2} + \frac{\kappa \sqrt{R^2 - Q^2 / \|\mathbf{g}_f\|^2}}{\|\mathbf{g}_f\|^2 \sqrt{\|\mathbf{g}_r\|^2 - \kappa^2 / \|\mathbf{g}_f\|^2}}.$$

Simplify the derivative further using $\kappa = \|\mathbf{g}_r\| \|\mathbf{g}_f\| \cos \theta$, where $\theta \in [0, \pi]$ is the angle between $\mathbf{g}_r$ and $\mathbf{g}_f$,

$$\frac{dF_r^*}{d\kappa} = \frac{1}{\|\mathbf{g}_f\|^2} \left(Q + \cot \theta \sqrt{R^2 \|\mathbf{g}_f\|^2 - Q^2} \right).$$

At the threshold $\kappa = \kappa_1 = -Q \|g_r\| / R$, the derivative evaluates to 0. When κ increases, for fixed $\|g_r\|$ and $\|g_f\|$, $\cos \theta$ increases, θ decreases, $\cot \theta$ increases. Hence, the derivative is positive and the function is monotonically increasing. $\square$

B.4. Unavoidable Collateral Forgetting (First-Order)

Theorem 4.4 is proven below.

Assume $Q \leq R \|g_f\|$. Problem (4) is infeasible if F_r^* (defined in Eq. (2)) is greater than 0. When $\kappa \leq \kappa_1$, $F_r^* = -R \|g_r\| < 0$ so Problem (4) is feasible. When $\kappa > \kappa_1$, the problem is infeasible if

$$\frac{\kappa Q}{\|g_f\|^2} > \sqrt{\left(\|g_r\|^2 - \frac{(g_r \cdot g_f)^2}{\|g_f\|^2}\right) \left(R^2 - \frac{Q^2}{\|g_f\|^2}\right)} \quad (9)$$

$$\begin{aligned} \kappa^2 Q^2 &> \left(\|g_r\|^2 \|g_f\|^2 - (g_r \cdot g_f)^2\right) \left(R^2 \|g_f\|^2 - Q^2\right) \\ \kappa^2 &> \|g_r\|^2 \|g_f\|^2 - \frac{Q^2 \|g_r\|^2}{R^2} \\ \kappa &> \sqrt{\|g_r\|^2 \|g_f\|^2 - \frac{Q^2 \|g_r\|^2}{R^2}}, \end{aligned} \quad (10)$$

where Ineq. (10) is because $\kappa > 0$, which is implied by Ineq. (9). This gives the local collateral forgetting condition.

When $\kappa < 0$, Ineq. (9) trivially fails; when $0 < \kappa \leq \sqrt{\|g_r\|^2 \|g_f\|^2 - \frac{Q^2 \|g_r\|^2}{R^2}}$, Ineq. (9) also fails. Therefore, the feasibility condition for Problem (4) is

$$\kappa \leq \sqrt{\|g_r\|^2 \|g_f\|^2 - \frac{Q^2 \|g_r\|^2}{R^2}}.$$

When $\kappa > 0$, let $\theta \in [0, \pi/2]$ be the angle between g_r and g_f , we can further simplify the feasibility condition:

$$\begin{aligned} \kappa = g_r \cdot g_f &\leq \sqrt{\|g_r\|^2 \|g_f\|^2 - \frac{Q^2 \|g_r\|^2}{R^2}} \\ \frac{g_r \cdot g_f}{\|g_r\| \|g_f\|} &\leq \frac{1}{\|g_r\| \|g_f\|} \sqrt{\|g_r\|^2 \|g_f\|^2 - \frac{Q^2 \|g_r\|^2}{R^2}} \\ \cos \theta &\leq \sqrt{1 - \frac{Q^2}{R^2 \|g_f\|^2}} \\ \frac{Q^2}{R^2 \|g_f\|^2} &\leq 1 - \cos^2 \theta \\ Q^2 &\leq R^2 \|g_f\|^2 \sin^2 \theta \\ Q &\leq R \|g_f\| \sin \theta. \end{aligned}$$

Problem (4) is always feasible when $\kappa \leq 0$; when $\kappa > 0$, let $\theta \in [0, \pi/2]$ be the angle between g_r and g_f , then Problem (4) is feasible if $Q \leq R \|g_f\| \sin \theta$, which is stricter than the feasibility condition of Problem (1) due to the additional constraint on retain utility.

B.5. Unavoidable Collateral Forgetting (Second-Order)

In this section, we discuss the local collateral forgetting condition in the second-order formulation. Identifying this condition requires solving the second-order formulation of Problem (1), which also yields an update rule but remains impractical due to assumptions and computational cost involving the Hessian. We find that it is also impossible to avoid collateral forgetting in the second-order formulation under certain scenarios. In fact, the hardness measure again plays an important role in determining whether we can avoid local collateral forgetting in machine unlearning. We show that **when the problem becomes too hard** ($\kappa = g_r \cdot g_f$ exceeds a certain threshold), **collateral forgetting is unavoidable**.

We formulate the problem similar to Problem (4), but better approximates $\Delta L_t(D_r)$ with a second-order approximation, while keeping first-order approximation for $\Delta L_t(D_f)$ to ensure we can still obtain the solution in analytical form. Assume the Hessian on the retain data $\mathbf{H}_r \triangleq \nabla_w^2 L_t(D_r)$ is positive definite, we wish (or prove impossible) to find $\Delta \mathbf{w}$ such that the following constraints are simultaneously satisfied:

$$\mathbf{g}_r \cdot \Delta \mathbf{w} + \frac{1}{2} \Delta \mathbf{w}^\top \mathbf{H}_r \Delta \mathbf{w} \leq 0 \quad \text{and} \quad \mathbf{g}_f \cdot \Delta \mathbf{w} \geq Q. \quad (11)$$

The first constraint requires the retain utility to not drop after the update, while the second constraint requires the forget quality to exceed a threshold $Q > 0$. Note that unlike in Problem (4), we do not constrain the maximum norm of the weight update, so if the problem is infeasible, then collateral forgetting is unavoidable for all possible weight updates $\Delta \mathbf{w}$. To find the infeasibility condition, we consider the following convex optimization problem, which is the second-order formulation of Problem (1):

$$\begin{aligned} \underset{\Delta \mathbf{w}}{\text{minimize}} \quad & F(\Delta \mathbf{w}) \triangleq \mathbf{g}_r \cdot \Delta \mathbf{w} + \frac{1}{2} \Delta \mathbf{w}^\top \mathbf{H}_r \Delta \mathbf{w} \\ \text{subject to} \quad & \mathbf{g}_f \cdot \Delta \mathbf{w} \geq Q. \end{aligned} \quad (12)$$

If the constraint in Problem (12) is satisfied, and the minimum objective value $F(\Delta \mathbf{w}^*) > 0$, then Problem (11) is infeasible. This establishes the local collateral forgetting condition in the second-order formulation. The Lagrangian of Problem (12) is given by

$$\mathcal{L}(\Delta \mathbf{w}, \gamma) = \mathbf{g}_r \cdot \Delta \mathbf{w} + \frac{1}{2} \Delta \mathbf{w}^\top \mathbf{H}_r \Delta \mathbf{w} + \gamma (Q - \mathbf{g}_f \cdot \Delta \mathbf{w}).$$

Since Problem (12) is a convex optimization problem with convex inequality constraints, we can find its optimal value by solving the KKT conditions:

Stationarity Condition	$\frac{\partial \mathcal{L}}{\partial \Delta \mathbf{w}} = \mathbf{g}_r + \mathbf{H}_r \Delta \mathbf{w} - \gamma \mathbf{g}_f = \mathbf{0}$
Complementary Slackness	$\gamma (Q - \mathbf{g}_f \cdot \Delta \mathbf{w}) = 0$
Primal Feasibility	$Q - \mathbf{g}_f \cdot \Delta \mathbf{w} \leq 0$
Dual Feasibility	$\gamma \geq 0.$

From the stationarity condition we have

$$\Delta \mathbf{w}^* = \gamma \mathbf{H}_r^{-1} \mathbf{g}_f - \mathbf{H}_r^{-1} \mathbf{g}_r. \quad (13)$$

Case 1: $\gamma = 0$.

Then $\Delta \mathbf{w}^* = -\mathbf{H}_r^{-1} \mathbf{g}_r$. We can verify that $\mathbf{g}_r \cdot \Delta \mathbf{w}^* + \frac{1}{2} \Delta \mathbf{w}^{*\top} \mathbf{H}_r \Delta \mathbf{w}^* \leq 0$ is always true:

$$\begin{aligned} & \mathbf{g}_r \cdot \Delta \mathbf{w}^* + \frac{1}{2} \Delta \mathbf{w}^{*\top} \mathbf{H}_r \Delta \mathbf{w}^* \\ &= \mathbf{g}_r \cdot (-\mathbf{H}_r^{-1} \mathbf{g}_r) + \frac{1}{2} (-\mathbf{H}_r^{-1} \mathbf{g}_r)^\top \mathbf{H}_r (-\mathbf{H}_r^{-1} \mathbf{g}_r) \\ &= -\mathbf{g}_r^\top \mathbf{H}_r^{-1} \mathbf{g}_r + \frac{1}{2} \mathbf{g}_r^\top \mathbf{H}_r^{-1} \mathbf{H}_r \mathbf{H}_r^{-1} \mathbf{g}_r \\ &= -\frac{1}{2} \mathbf{g}_r^\top \mathbf{H}_r^{-1} \mathbf{g}_r \\ &\leq 0, \end{aligned}$$

where we use $(\mathbf{H}_r^{-1})^\top = (\mathbf{H}_r^\top)^{-1} = \mathbf{H}_r^{-1}$ because $\mathbf{H}_r$ is symmetric, and the inequality is due to $\mathbf{H}_r \succ 0$. Therefore, Problem (11) is feasible if $\Delta \mathbf{w}^*$ additionally satisfies $\mathbf{g}_f \cdot \Delta \mathbf{w}^* \geq Q$. To simplify discussion, we first assume a spherical

loss landscape, i.e., $\mathbf{H}_r = \lambda \mathbf{I}$ for some $\lambda > 0$, and discuss the general case afterwards. Problem (11) is infeasible if

$$\begin{aligned} \mathbf{g}_f \cdot \Delta \mathbf{w}^* &< Q \\ \mathbf{g}_f \cdot (-\mathbf{H}_r^{-1} \mathbf{g}_r) &< Q \\ -\mathbf{g}_f^\top \mathbf{H}_r^{-1} \mathbf{g}_r &< Q \\ -\mathbf{g}_f^\top \left(\frac{1}{\lambda} \mathbf{I} \right) \mathbf{g}_r &< Q \\ \mathbf{g}_r \cdot \mathbf{g}_f &> -\lambda Q. \end{aligned}$$

For the infeasibility condition of general $\mathbf{H}_r \succ 0$, we first establish a lower bound for $\mathbf{g}_f^\top \mathbf{H}_r^{-1} \mathbf{g}_r$ with the following lemma:

Lemma B.1. *Let $\lambda_{\min}, \lambda_{\max}$ be the minimum and maximum eigenvalues of a symmetric matrix $\mathbf{A}_{d \times d}$, and $\mathbf{u}, \mathbf{v}$ are vectors of dimension d , then $\mathbf{u}^\top \mathbf{A} \mathbf{v} \geq \lambda_{\min} \mathbf{u} \cdot \mathbf{v} - (\lambda_{\max} - \lambda_{\min}) \|\mathbf{u}\| \|\mathbf{v}\|$.*

Proof. Decompose $\mathbf{A}$ into $\mathbf{A} = \lambda_{\min} \mathbf{I} + (\mathbf{A} - \lambda_{\min} \mathbf{I})$ and denote $\mathbf{B} \triangleq \mathbf{A} - \lambda_{\min} \mathbf{I}$. Then the spectral norm of $\mathbf{B}$ is $\|\mathbf{B}\|_2 = \lambda_{\max} - \lambda_{\min}$. We have

$$\begin{aligned} \mathbf{u}^\top \mathbf{A} \mathbf{v} &= \mathbf{u}^\top (\lambda_{\min} \mathbf{I}) \mathbf{v} + \mathbf{u}^\top \mathbf{B} \mathbf{v} \\ &\geq \lambda_{\min} \mathbf{u} \cdot \mathbf{v} - \|\mathbf{u}\| \|\mathbf{B} \mathbf{v}\| && \text{(Cauchy-Schwarz inequality)} \\ &\geq \lambda_{\min} \mathbf{u} \cdot \mathbf{v} - \|\mathbf{u}\| \|\mathbf{B}\|_2 \|\mathbf{v}\| && \text{(Definition of spectral norm)} \\ &= \lambda_{\min} \mathbf{u} \cdot \mathbf{v} - (\lambda_{\max} - \lambda_{\min}) \|\mathbf{u}\| \|\mathbf{v}\|. \end{aligned}$$

□

Problem (11) is infeasible if $-\mathbf{g}_f^\top \mathbf{H}_r^{-1} \mathbf{g}_r < Q$, by applying Lemma B.1 we obtain the infeasibility condition:

$$\mathbf{g}_r \cdot \mathbf{g}_f > \frac{1}{\lambda_{\min}} [(\lambda_{\max} - \lambda_{\min}) \|\mathbf{g}_r\| \|\mathbf{g}_f\| - Q],$$

where $\lambda_{\min}, \lambda_{\max}$ are the minimum and maximum eigenvalues of $\mathbf{H}_r^{-1}$, respectively.

Case 2: $\gamma > 0$.

Primal feasibility must bind so $\mathbf{g}_f \cdot \Delta \mathbf{w}^* = Q$. Substituting the expression for $\Delta \mathbf{w}^*$ in Eq. (13) and solving for γ ,

$$\gamma = \frac{Q + \mathbf{g}_f^\top \mathbf{H}_r^{-1} \mathbf{g}_r}{\mathbf{g}_f^\top \mathbf{H}_r^{-1} \mathbf{g}_f}.$$

To simplify notation, let $A = \mathbf{g}_f^\top \mathbf{H}_r^{-1} \mathbf{g}_f$, $B = \mathbf{g}_r^\top \mathbf{H}_r^{-1} \mathbf{g}_r$, $C = \mathbf{g}_r^\top \mathbf{H}_r^{-1} \mathbf{g}_f = \mathbf{g}_f^\top \mathbf{H}_r^{-1} \mathbf{g}_r$, then $\gamma = \frac{Q+C}{A}$, and

$$\begin{aligned} F(\Delta \mathbf{w}^*) &= \mathbf{g}_r \cdot (\gamma \mathbf{H}_r^{-1} \mathbf{g}_f - \mathbf{H}_r^{-1} \mathbf{g}_r) + \frac{1}{2} (\gamma \mathbf{H}_r^{-1} \mathbf{g}_f - \mathbf{H}_r^{-1} \mathbf{g}_r)^\top \mathbf{H}_r (\gamma \mathbf{H}_r^{-1} \mathbf{g}_f - \mathbf{H}_r^{-1} \mathbf{g}_r) \\ &= \gamma C - B + \frac{1}{2} (\gamma^2 A - 2\gamma C + B) \\ &= \frac{1}{2} \gamma^2 A - \frac{1}{2} B \\ &= \frac{1}{2} \left(\frac{Q+C}{A} \right)^2 A - \frac{1}{2} B \\ &= \frac{1}{2} \frac{(Q+C)^2}{A} - \frac{1}{2} B. \end{aligned}$$

Problem (11) is infeasible if $F(\Delta \mathbf{w}^*) > 0$, which is equivalent to $(Q+C)^2 > AB$. Note that $\frac{Q+C}{A} = \gamma > 0$ and $A > 0$ since $\mathbf{H}_r^{-1} \succ 0$ (assuming $\mathbf{g}_f \neq \mathbf{0}$), we know that $Q+C > 0$. The infeasibility condition then becomes

$$Q + C > \sqrt{AB}. \quad (14)$$

If we again assume that $\mathbf{H}_r = \lambda \mathbf{I}$ for some $\lambda > 0$, then $A = \frac{1}{\lambda} \|\mathbf{g}_f\|^2$, $B = \frac{1}{\lambda} \|\mathbf{g}_r\|^2$, $C = \frac{1}{\lambda} \mathbf{g}_r \cdot \mathbf{g}_f$. Thus,

$$\begin{aligned} Q + \frac{1}{\lambda} \mathbf{g}_r \cdot \mathbf{g}_f &> \sqrt{\frac{1}{\lambda} \|\mathbf{g}_r\|^2 \cdot \frac{1}{\lambda} \|\mathbf{g}_f\|^2} \\ \mathbf{g}_r \cdot \mathbf{g}_f &> \|\mathbf{g}_r\| \|\mathbf{g}_f\| - \lambda Q. \end{aligned} \quad (15)$$

For the general case, again let $\lambda_{\min}, \lambda_{\max}$ be the minimum and maximum eigenvalues of $\mathbf{H}_r^{-1}$, by applying Lemma B.1,

$$C = \mathbf{g}_r^\top \mathbf{H}_r^{-1} \mathbf{g}_f \geq \lambda_{\min} \mathbf{g}_r \cdot \mathbf{g}_f - (\lambda_{\max} - \lambda_{\min}) \|\mathbf{g}_r\| \|\mathbf{g}_f\|.$$

By the Rayleigh quotient bound, $A \leq \lambda_{\max} \|\mathbf{g}_f\|^2$, $B \leq \lambda_{\max} \|\mathbf{g}_r\|^2$, thus

$$\sqrt{AB} - Q \leq \sqrt{\lambda_{\max} \|\mathbf{g}_f\|^2 \cdot \lambda_{\max} \|\mathbf{g}_r\|^2} - Q = \lambda_{\max} \|\mathbf{g}_r\| \|\mathbf{g}_f\| - Q.$$

From Ineq. (14) we know the infeasibility condition is

$$C > \sqrt{AB} - Q.$$

Therefore, the problem is infeasible if the lower bound of C is strictly greater than the upper bound of $\sqrt{AB} - Q$,

$$\begin{aligned} \lambda_{\min} \mathbf{g}_r \cdot \mathbf{g}_f - (\lambda_{\max} - \lambda_{\min}) \|\mathbf{g}_r\| \|\mathbf{g}_f\| &> \lambda_{\max} \|\mathbf{g}_r\| \|\mathbf{g}_f\| - Q \\ \lambda_{\min} \mathbf{g}_r \cdot \mathbf{g}_f &> 2\lambda_{\max} \|\mathbf{g}_r\| \|\mathbf{g}_f\| - \lambda_{\min} \|\mathbf{g}_r\| \|\mathbf{g}_f\| - Q \\ \mathbf{g}_r \cdot \mathbf{g}_f &> \frac{1}{\lambda_{\min}} [(2\lambda_{\max} - \lambda_{\min}) \|\mathbf{g}_r\| \|\mathbf{g}_f\| - Q]. \end{aligned}$$

Observe that **we can combine both cases** since the condition when $\lambda = 0$ is subsumed by the condition when $\lambda > 0$:

$$\frac{1}{\lambda_{\min}} [(2\lambda_{\max} - \lambda_{\min}) \|\mathbf{g}_r\| \|\mathbf{g}_f\| - Q] \geq \frac{1}{\lambda_{\min}} [(\lambda_{\max} - \lambda_{\min}) \|\mathbf{g}_r\| \|\mathbf{g}_f\| - Q],$$

and

$$\|\mathbf{g}_r\| \|\mathbf{g}_f\| - \lambda Q \geq -\lambda Q.$$

We interpret the local collateral forgetting condition under the assumption of a spherical loss landscape (Ineq. (15)). The maximum dot product $\|\mathbf{g}_r\| \|\mathbf{g}_f\|$ describes the maximum hardness of the problem, the condition says that if the actual hardness of the problem is too close to this maximum, then collateral forgetting is unavoidable. The term $-\lambda Q$ acts as a safety margin, lowering which lowers the threshold and it is easier for the problem to become infeasible. If λ is large, the loss surface is steep, it is harder to bound the decrease in retain utility. If Q is large, we are putting a stricter demand on the forget quality, which makes it harder to satisfy both constraints simultaneously. In the extreme case when λQ is large enough, it is possible that $\|\mathbf{g}_r\| \|\mathbf{g}_f\| - \lambda Q < 0$, and even a negative $\mathbf{g}_r \cdot \mathbf{g}_f$ could theoretically be infeasible.

When the problem is feasible, i.e., both constraints are satisfied, Eq. (13) provides an unlearning update that does not induce collateral forgetting. However, it requires computing and inverting the Hessian, which is prohibitively expensive. It also requires the additional assumption that the Hessian is positive definite, hence limiting its applicability to non-convex models, whereas our first-order algorithm can be applied to any non-convex model as long as a suitable R is chosen for approximation quality.

B.6. The Retain-Constrained Problem

The retain-constrained problem provides an alternative formulation to Problem (1). In Problem (16), we optimize for the best forget quality improvement while also enforcing a constraint on the minimum retain utility improvement. The retain-constrained problem is formulated below for $U > 0$, $R > 0$:

$$\begin{aligned} &\underset{\Delta \mathbf{w}}{\text{maximize}} && \mathbf{g}_f \cdot \Delta \mathbf{w} \\ &\text{subject to} && \mathbf{g}_r \cdot \Delta \mathbf{w} \leq -U, \quad \|\Delta \mathbf{w}\| \leq R. \end{aligned} \quad (16)$$

which is the same as

$$\begin{aligned} & \underset{\Delta \mathbf{w}}{\text{minimize}} && -\mathbf{g}_f \cdot \Delta \mathbf{w} \\ & \text{subject to} && -\mathbf{g}_r \cdot \Delta \mathbf{w} \geq U, \quad \|\Delta \mathbf{w}\| \leq R. \end{aligned}$$

This is structurally equivalent to Problem (1), and is only feasible when $U \leq R \|\mathbf{g}_r\|$. We can derive the solution to Problem (16) by substituting $\mathbf{g}_r \leftarrow -\mathbf{g}_f, \mathbf{g}_f \leftarrow -\mathbf{g}_r, Q \leftarrow U$ into the solution to Problem (1), and then negating the optimal value. Note that the hardness measure $\kappa = \mathbf{g}_r \cdot \mathbf{g}_f$ is preserved under this substitution:

$$\begin{aligned} \Delta \mathbf{w}^* &= \begin{cases} -\frac{R}{\|\mathbf{g}_f\|} \cdot (-\mathbf{g}_f) & \text{if } \kappa \leq -\frac{U \|\mathbf{g}_f\|}{R} \\ \frac{U}{\|\mathbf{g}_r\|^2} \cdot (-\mathbf{g}_r) - \sqrt{R^2 - \frac{U^2}{\|\mathbf{g}_r\|^2}} \cdot \frac{-\mathbf{g}_f \perp}{\|\mathbf{g}_f \perp\|} & \text{otherwise} \end{cases} \\ &= \begin{cases} \frac{R}{\|\mathbf{g}_f\|} \cdot \mathbf{g}_f & \text{if } \kappa \leq \kappa_3 \\ -\frac{U}{\|\mathbf{g}_r\|^2} \cdot \mathbf{g}_r + \sqrt{R^2 - \frac{U^2}{\|\mathbf{g}_r\|^2}} \cdot \frac{\mathbf{g}_f \perp}{\|\mathbf{g}_f \perp\|} & \text{otherwise,} \end{cases} \end{aligned}$$

where $\mathbf{g}_f \perp \triangleq \mathbf{g}_f - \frac{\mathbf{g}_r \cdot \mathbf{g}_f}{\|\mathbf{g}_r\|^2} \cdot \mathbf{g}_r$ and $\kappa_3 \triangleq -U \|\mathbf{g}_f\| / R$. Similarly, we have

$$\min(-\mathbf{g}_f \cdot \Delta \mathbf{w}) = \begin{cases} -R \|\mathbf{g}_f\| & \text{if } \kappa \leq \kappa_3 \\ \frac{\kappa U}{\|\mathbf{g}_r\|^2} - \sqrt{\left(\|\mathbf{g}_f\|^2 - \frac{\kappa^2}{\|\mathbf{g}_r\|^2}\right) \left(R^2 - \frac{U^2}{\|\mathbf{g}_r\|^2}\right)} & \text{otherwise.} \end{cases}$$

The optimal value of the original problem is then its negation:

$$\max \mathbf{g}_f \cdot \Delta \mathbf{w} = \begin{cases} R \|\mathbf{g}_f\| & \text{if } \kappa \leq \kappa_3 \\ -\frac{\kappa U}{\|\mathbf{g}_r\|^2} + \sqrt{\left(\|\mathbf{g}_f\|^2 - \frac{\kappa^2}{\|\mathbf{g}_r\|^2}\right) \left(R^2 - \frac{U^2}{\|\mathbf{g}_r\|^2}\right)} & \text{otherwise.} \end{cases}$$

This also implies that $\max \mathbf{g}_f \cdot \Delta \mathbf{w}$ monotonically decreases with κ for fixed $\|\mathbf{g}_r\|$ and $\|\mathbf{g}_f\|$, given $F_r^* = \min \mathbf{g}_r \cdot \Delta \mathbf{w}$ monotonically increases with κ for fixed $\|\mathbf{g}_r\|$ and $\|\mathbf{g}_f\|$, and the fact that hardness κ is preserved under our substitution.

The direct update ($\kappa \leq \kappa_3$) in the retain-constrained problem corresponds to gradient ascent on the forget data if we set $R = \eta \|\mathbf{g}_f\|$. To find the stopping criterion of HAMU-U, we find the infeasibility condition to the following problem:

$$\mathbf{g}_f \cdot \Delta \mathbf{w} \geq 0, \quad \mathbf{g}_r \cdot \Delta \mathbf{w} \leq -U \quad \text{and} \quad \|\Delta \mathbf{w}\| \leq R. \quad (17)$$

Similar to App. B.4, we cannot find a feasible $\Delta \mathbf{w}$ to Problem (17) if $\max \mathbf{g}_f \cdot \Delta \mathbf{w} < 0$. The stopping criterion is obtained by solving the inequality. Consequently, we terminate HAMU-U when $\kappa > \kappa_4$, where

$$\kappa_4 \triangleq \sqrt{\|\mathbf{g}_r\|^2 \|\mathbf{g}_f\|^2 - \frac{U^2 \|\mathbf{g}_f\|^2}{R^2}}.$$

We provide the pseudocode to HAMU-U in Alg. 2.

C. Adapting Stopping Criterion for Layer-Specific Constraints

In Alg. 1, the stopping criterion is specified to be

$$\mathbf{g}_r \cdot \mathbf{g}_f = \kappa > \kappa_2 = \sqrt{(\|\mathbf{g}_r\| \|\mathbf{g}_f\|)^2 - \frac{Q^2}{R^2} \|\mathbf{g}_r\|^2}.$$

Algorithm 2 HAMU-U

```

1: Input: initial weights  $w_0$ , retain data  $D_r$ , forget data  $D_f$ , batch size  $m$ , learning rate  $\eta$ , maximum iterations  $T$ , retain
   quality constraint  $U > 0$ 
2: for  $t = 0, 1, \dots, T - 1$  do
3:   Sample i.i.d. size- $m$  batches  $B_r \sim D_r$  and  $B_f \sim D_f$ 
4:    $\bar{g}_r \leftarrow \nabla_w L_t(B_r)$ ;  $\bar{g}_f \leftarrow \nabla_w L_t(B_f)$ 
5:    $R \leftarrow \eta \|\bar{g}_f\|$ 
6:   if  $U > R \|\bar{g}_r\|$  then
7:     break
8:   end if
9:    $\bar{\kappa} \leftarrow \bar{g}_r \cdot \bar{g}_f$ 
10:   $\bar{\kappa}_3 \leftarrow -U \|\bar{g}_f\| / R$ 
11:   $\bar{\kappa}_4 \leftarrow \sqrt{(\|\bar{g}_r\| \|\bar{g}_f\|)^2 - U^2 \|\bar{g}_f\|^2 / R^2}$ 
12:  if  $\bar{\kappa} > \bar{\kappa}_4$  then
13:    break
14:  end if
15:  if  $\bar{\kappa} \leq \bar{\kappa}_3$  then
16:     $\Delta w \leftarrow \frac{R}{\|\bar{g}_f\|} \cdot \bar{g}_f$ 
17:  else
18:     $\bar{g}_{f\perp} \leftarrow \bar{g}_f - \frac{\bar{\kappa}}{\|\bar{g}_r\|^2} \cdot \bar{g}_r$ 
19:     $\Delta w \leftarrow -\frac{U}{\|\bar{g}_r\|^2} \cdot \bar{g}_r + \sqrt{R^2 - \frac{U^2}{\|\bar{g}_r\|^2}} \cdot \frac{\bar{g}_{f\perp}}{\|\bar{g}_{f\perp}\|}$ 
20:  end if
21:   $w_{t+1} \leftarrow w_t + \Delta w$ 
22: end for
23: return  $w_t$ 

```

This inequality is equivalent to

$$Q > \frac{R}{\|\mathbf{g}_r\|} \sqrt{(\|\mathbf{g}_r\| \|\mathbf{g}_f\|)^2 - (\mathbf{g}_r \cdot \mathbf{g}_f)^2}. \quad (18)$$

As described in Sec. 5.1, the global constraint Q can be distributed into layer-wise constraints $(Q_i)_{i=1}^\ell$, and the stopping criterion has to be adapted accordingly. One possible way of terminating the unlearning process could be to replace all variable in Ineq. (18) with their layer-wise counterparts,

$$Q_i > \frac{R}{\|\mathbf{g}_r^{(i)}\|} \sqrt{(\|\mathbf{g}_r^{(i)}\| \|\mathbf{g}_f^{(i)}\|)^2 - (\mathbf{g}_r^{(i)} \cdot \mathbf{g}_f^{(i)})^2}, \quad (19)$$

such that unlearning terminates when this layer-wise stopping criterion is met for any layer i in the model.

However, as described in Sec. 5.1, different layers in the model can exhibit varying sensitivities to the same data. It might be harder to unlearn at certain layers compared to other layers, such that the layer-wise stopping criterion in Ineq. (19) is triggered in an earlier unlearning epoch for layers that are harder to unlearn. Terminating when any layer meets this criterion would result in an undesirably early termination of unlearning, when other layers have not yet met the criterion.

Instead, considering the layer-wise constraints are distributed with $\sum_{i=1}^\ell Q_i = Q$, we opt to terminate when this aggregate condition across all layers is met,

$$Q = \sum_{i=1}^\ell Q_i > \sum_{i=1}^\ell \frac{R}{\|\mathbf{g}_r^{(i)}\|} \sqrt{(\|\mathbf{g}_r^{(i)}\| \|\mathbf{g}_f^{(i)}\|)^2 - (\mathbf{g}_r^{(i)} \cdot \mathbf{g}_f^{(i)})^2}. \quad (20)$$

Using Ineq. (20) as the stopping criterion allows layers that are easier to unlearn to compensate for layers that are harder to unlearn, such that unlearning only terminates when the required forget quality threshold exceeds the value that can be sustained with the aggregate of all layers in the model. This is the stopping criterion that we have used in Fig. 9.

D. Unlearning Algorithms

In this paper, we compare HAMU against the following loss-based and gradient-based unlearning algorithms. We use the same learning rate of $1e-4$ for all unlearning algorithms in our experiments.

Gradient ascent (GA) The gradient ascent unlearning algorithm (Jang et al., 2023) only trains on the forget data, and uses a negated loss function to increase the loss on the forget data.

Fine-tune (FT) The fine-tune unlearning algorithm trains the model using only the retain data, to decrease the loss on the retain data.

Gradient difference (GDiff) The gradient difference unlearning algorithm (Liu et al., 2022) combines GA and FT, with gradient ascent on the forget data and gradient descent on the retain data.

KL minimization (KL) The KL minimization algorithm (Maini et al., 2024) combines gradient ascent on the forget data with minimizing the Kullback-Leibler divergence between the prediction of the model being unlearned and the original model before unlearning on the retain data.

SCRUB The SCRUB algorithm (Kurmanji et al., 2023) uses a weighted combination of different losses, by maximizing the KL divergence between the predictions of the unlearned model and original model on the forget data, while minimizing the loss and KL divergence of the predictions on the retain data. Following the source code⁶ of SCRUB, we set $\alpha = 0.001$ and $\gamma = 0.99$.

Gradient Rectified Unlearning (GRU) The Gradient Rectified Unlearning (GRU) algorithm (Wang et al., 2025b) projects the gradient on the forget data, $\mathbf{g}_f$, such that the new vector is orthogonal to the gradient on the retain data, $\mathbf{g}_r$, when the angle between the two gradients is obtuse.

Projecting Conflicting Gradients (PCGrad) We adapt the projecting conflicting gradients algorithm (Yu et al., 2020) to our unlearning setting. If the angle between $\mathbf{g}_f$ and $\mathbf{g}_r$ is obtuse, $\mathbf{g}_f$ is projected to be orthogonal to $\mathbf{g}_r$ and vice versa for $\mathbf{g}_r$, and the mean of the two projected gradients is used for the weight update.

HAMU For our unlearning algorithm HAMU, we perform weight updates according to Alg. 1 and Alg. 2.

D.1. Implementation Details

For unlearning algorithms that use both the retain and forget data (HAMU, GDiff, KL, SCRUB), we duplicate the forget data to match the size of the retain data. For unlearning algorithms which only use the retain data (FT) or the forget data (GA), we duplicated smaller unlearning datasets to match the size of the dataset used when using both retain and forget data (HAMU, GDiff, KL, SCRUB). For fair comparison, all algorithms were re-implemented with the same training code, with only the specific weight update step swapped out for each unlearning algorithm. The same GPU optimizations described in Sec. G.1 were used for all methods which require gradients of both the forget and retain data (GDiff, KL, SCRUB, GRU, PCGrad, HAMU). This provides an additional advantage for other unlearning methods, and might result in lower resource requirements than those reported in the original papers. In our experiments, we ran all unlearning algorithms using the same learning rate for the same number of unlearning steps on the same hardware and number of GPUs. Unless otherwise stated, we use 42 as the random seed in all experiments.

⁶<https://github.com/meghdadk/SCRUB>

E. CV Experimental Setup

For training and unlearning of models for CV task, our code is adapted from the Huggingface Trainer⁷.

E.1. Dataset Construction

We used the CIFAR-10 dataset, which consists of 50000 training samples, with 10 classes of equal size. No pre-processing transformations (scale, flip or zoom) were used on the training dataset.

To construct the forget data, we first choose a random class c which contains n samples. Given the parameter $\rho \in [0, 1]$, the forget data consists of $n - \lfloor \rho n \rfloor$ randomly sampled data points from class c , and $\lfloor \rho n \rfloor$ randomly sampled data points from the union of all other classes and data points in class c that are not selected in the first step.

- $\rho = 0$: forget one entire class, easy instance
- $\rho = 1$: forget i.i.d. samples from all classes, hard instance

In our experiments, we used $c = 0$ and $\rho \in \{0, 0.25, 0.5, 0.75, 1\}$.

E.2. Training of CV Model

We used the ResNet-20 architecture (He et al., 2016) with randomly initialized weights. We used a fixed learning rate of $1e-3$ with the AdamW optimizer (Loshchilov & Hutter, 2019), batch size of 5000 and trained for 50 epochs. Every 5 epochs, we performed evaluation on the CIFAR-10 test set, which consists of 10000 samples across the 10 classes, and saved the checkpoint model weights. The final trained model was selected to be the checkpoint with the highest average retain and forget data evaluation accuracy. Training was done on 1 NVIDIA L40 GPU, requiring approximately 10 minutes for each model.

E.3. Unlearning of CV Model

To perform unlearning, we used a batch size of 5000 samples per GPU, and trained for a maximum of 25 epochs. We used a fixed learning rate or $1e-4$ for all unlearning algorithms, without any momentum-based optimizers. Additional discussion and results with the use of optimizers are in App. G. Unlearning was done on 2 NVIDIA L40 GPUs, with each run of the unlearning experiment requiring approximately 10 minutes (assuming no early termination). The unlearning dataset is shuffled once before the start of unlearning, and is not shuffled between unlearning epochs.

F. LLM Experimental Setup

For LLM training and unlearning, our code is adapted from the Supervised Fine-Tuning (SFT) Trainer⁸.

F.1. Dataset Construction

We used the WaterDrum-TOFU unlearning dataset for the LLM experiments, which consists of QA pairs for an LLM QA task. This dataset extends the TOFU unlearning dataset (Maini et al., 2024) by constructing different variations of the QA pairs of different levels of similarity, watermarked with the text watermarking scheme Waterfall (Lau et al., 2024). This dataset contains different splits, of which we used the watermarked “No duplicate” and “Semantic duplicate” splits for 10% of the forget data. We refer to the settings using these two splits as “semantically dissimilar” and “semantically similar”.

Following the implementation in Lu et al. (2026), the “semantically dissimilar” setting consists of 400 samples in the forget data, and 3600 samples in the retain data, while the “semantically similar” setting consists of 400 samples in the forget data, and 4000 samples in the retain data, of which 400 samples in the retain data are paraphrased versions of the 400 samples in the forget data.

⁷https://huggingface.co/docs/transformers/en/main_classes/trainer

⁸https://huggingface.co/docs/trl/main/en/sft_trainer

F.2. Training of LLM

The original full LLM was trained from the base LLM `Llama-2-7b-chat-hf`. We used LoRA (Hu et al., 2022) with $r = 8$ and $\alpha = 16$, batch size of 64 and trained for 10 epochs. We used a learning rate of $1e-4$ with the AdamW optimizer (Loshchilov & Hutter, 2019) during LLM training. Training was done on 1 NVIDIA H200 GPU, requiring approximately 20 minutes for each LLM.

F.3. Unlearning of LLM

We used a batch size of 50 samples per GPU, and trained for a maximum of 720 steps (equivalent of 10 epochs on D_r). We performed unlearning without using any momentum-based optimizers. Unlearning was done on 2 NVIDIA H200 GPUs, with each run of the unlearning experiment requiring approximately 20 minutes (assuming no early termination). The unlearning dataset is shuffled once before the start of unlearning, and is not shuffled between unlearning epochs.

For this experiment, we used the `WaterDrum-TOFU` dataset with 10% of the training dataset being the forget data, where the retain data is 9 times the size of the forget data. When only using the retain data (e.g., FT), we duplicated the retain data 2 times. When only using the forget data (e.g., GA), we duplicated the forget data 18 times.

In the experiments, we terminated HAMU-Q once the feasibility criterion $Q \leq R \|g_f\|$ of Problem (1) fails. Additionally, to reduce the total time required for all experiments, we implemented some early termination checks in the unlearning process. We terminate the algorithm when forget loss decreases below 0.1 (model is not unlearning on the forget data), or retain loss increases above 0.4 (model has unlearned too much on the retain data). For reference, the losses on the forget and retain data are both approximately 0.3 before unlearning.

G. Practical Considerations with Modern Deep Learning Frameworks

G.1. Parallelized Computation of Forget and Retain Gradients

As our proposed unlearning algorithm requires the use of and comparison between the gradients of the retain samples $\bar{g}_r$ and gradients of the forget samples $\bar{g}_f$, both copies of the gradients are required to be available on the GPU at each iteration. A naive approach at each step would be to perform the forward and backward pass on the retain samples, store the gradients separately, then perform the forward and backward pass on the forget samples, and finally compare the two gradients. This incurs a high memory cost on the GPU, as memory has to be allocated for both $\bar{g}_r$ and $\bar{g}_f$. As such, we would only be able to process half the number of samples at each iteration as compared to if we only needed one copy of the gradients based on the memory available on the GPU. This is also true even if this naive approach is used with multi-GPU model parallelism, where each GPU still has to allocate memory for two copies of gradients.

To perform this efficiently, we implemented a parallelized unlearning script, which allocates the forget and retain samples to alternate GPUs when ran on an even number of GPUs. This allows half of the GPUs to perform computation on the retain samples while the other half of the GPUs perform computation on the forget samples at the same time. Each half of the GPUs would only require memory to store either $\bar{g}_r$ or $\bar{g}_f$ (but not both), allowing us to make full use of compute and memory resource on all GPUs. As each GPU only needs enough memory to store a single copy of its own gradients, we would effectively be able to process double the batch size with the same memory available, as compared to the naive implementation (or enable HAMU to be used when two copies of the gradients cannot fit within the memory of a single GPU).

To implement this, we made use of the integration between the SFT Trainer and `accelerate` library (Gugger et al., 2022) for a multi-GPU setup. During the data preparation step, we interleave the forget and retain samples into the unlearning dataset according to the batch size of each GPU, such that the dataloader would send forget samples to one half of the GPUs and retain sample to the other GPUs. During the backward pass, instead of simply aggregating the gradients from all GPUs as in a typical multi-GPU setup, we instead aggregate the gradients from alternate GPUs responsible for the forget and retain samples separately. When comparison of $\bar{g}_r$ and $\bar{g}_f$ are required, each layer’s gradient is iteratively sent over to the other GPUs as needed, such that only a small amount of additional memory is required to store only the single layer’s worth of gradients. These gradients are then passed into the weight update function which computes the final weight update according to the unlearning algorithm. Exact implementation is in our provided code.

Table 1 shows the GPU memory and computation cost of the different unlearning methods under the CV experimental setting (App. E). All methods are evaluated under the same hardware and process the same number of samples. Reported

Table 1. Average computational time per epoch and peak GPU memory usage of unlearning across the different unlearning methods under the CV setting.

Method	FT	GA	GDiff	KL	SCRUB	HAMU-Q	HAMU-U
Unlearning time (seconds/epoch)	0.879	0.884	0.868	1.067	1.064	0.976	0.994
Peak GPU memory (GB)	7.65	7.65	7.65	8.87	8.87	7.65	7.65

timing excludes computations other than unlearning, such as script startup, evaluation and logging operations. Results show HAMU only incurs modest computational overhead compared to standard baselines, and remains comparable to other multi-objective methods (KL, SCRUB). For memory, HAMU does not incur additional GPU memory as it only fetches gradients from other GPU when needed and performs computation in-place, whereas KL and SCRUB require more GPU memory to store a copy of the original model.

G.2. Using HAMU with Optimizers

In the main paper, our experiments are set up in a way to clearly demonstrate the theoretical formulation in Problem (1) and Problem (16). However, when deploying unlearning algorithms in practice on real world ML models, there are often other practical constraints and requirements, as well as practical optimizations that could be made to improve the overall performance. For practical constraints, the model owner might be limited to a small number of unlearning steps (e.g., only 1 epoch) such that the computational cost of unlearning remains small relative to retraining. For practical optimizations, the model owner typically implements momentum-based optimizers (e.g., AdamW (Loshchilov & Hutter, 2019)) to accelerate the convergence of the optimization process.

Optimizers such as AdamW (Loshchilov & Hutter, 2019) speed up the optimization process by incorporating the weight updates in previous iterations to the update in the current iteration. When the gradients do not change significantly between iterations, the step size taken by each weight update gradually increases. This speeds up the unlearning process, and the model is able to achieve a similar or better forget quality and retain utility with fewer number of unlearning steps.

To allow our weight update to work with momentum-based optimizers, instead of directly performing weight update with $w_{t+1} = w_t + \Delta w^*$, we instead compute the equivalent gradient which would have resulted in a gradient descent weight update, $g^* = -\frac{\Delta w^*}{\eta}$. This equivalent gradient g^* is then passed to the momentum-based optimizers. When we set $R = \eta \|g_r\|$, the equivalent gradient for HAMU-Q is

$$g^* = \begin{cases} g_r & \text{if } \kappa \leq \kappa_1 \\ -\frac{Q \|g_r\|}{R \|g_f\|} \cdot g_f + \sqrt{\|g_r\|^2 - \left(\frac{Q \|g_r\|}{R \|g_f\|}\right)^2} \cdot \frac{g_{r\perp}}{\|g_{r\perp}\|} & \text{otherwise.} \end{cases} \quad (21)$$

Fig. 12 shows that HAMU is compatible with the use of optimizers, and allows for unlearning to be performed with fewer number of unlearning steps. Using AdamW with only 1 epoch achieves a higher ΔL_f and $-\Delta L_r$ than vanilla SGD in 5 epochs.

G.3. Sensitivity analysis for unlearning batch size

In our experimental setup, we used a large batch size that could still fit within the available memory in the GPU. However, there will be situations where a smaller batch size has to be used, such as when a GPU with less memory is used, the model being unlearned is large with more parameters, or text training data for LLMs has longer sequence lengths. A smaller batch size can result in several possible issues, such as poorer approximation and high variance between iterations of batch gradient estimates when updating the weights.

To resolve these issues, we have implemented several possible solutions in our code. Firstly, our code can be run with more than 2 GPUs, such that alternate GPUs handle the gradient computation of the retain and forget data respectively. When more GPUs are available, it results in a larger effective batch size. Secondly, our code is also fully compatible with gradient accumulation, where gradient values are collated across multiple iterations of the unlearning loop before the weight update is performed, similarly achieving a larger effective batch size. Lastly, to address the issue of fluctuations in $\bar{\kappa}$ prematurely triggering the stopping criterion, practical adjustments can be made to the stopping condition, such as implementing a

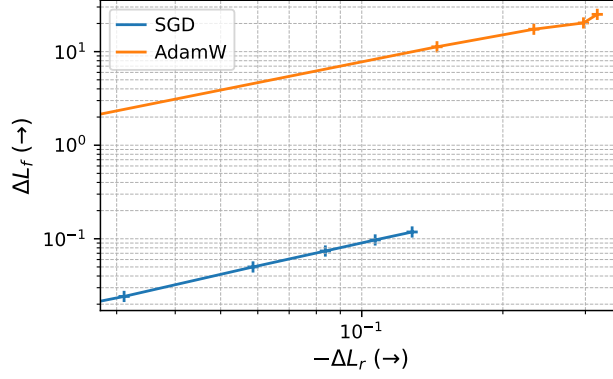


Figure 12. Comparing unlearning trajectory when using HAMU-U ($Q = 0.9\eta$) with vanilla SGD versus AdamW, for 5 unlearning epochs. Note that the axes are plotted in log scale. A larger increase in ΔL_f (and $-\Delta L_r$) is achieved with the same number of unlearning epochs for AdamW than vanilla SGD.

moving average of κ/κ_2 , or only stop when the stopping criterion is met for a certain number of times.

To evaluate the effect of different batch size on HAMU, we conducted ablation experiments, varying the batch size used in the CV experiments. Table 2 shows the unlearning performance of HAMU-Q with batch size between 128 and 2048. We observe that smaller batch sizes tend to achieve better forget quality, likely due to more frequent weight updates for a fixed number of training samples (a smaller batch size will have more weight update steps within a single epoch). Retain utility is relatively robust to batch size, but very small batches (128) degrade the retain utility, and increase runtime due to poor GPU utilization. Under this experimental setting, batch size 512 provides a good balance.

Table 2. Sensitivity analysis with varying batch size.

Batch size	$-\Delta L_r \uparrow$	$\Delta L_f \uparrow$	Retain accuracy (%) $\uparrow$	Forget accuracy (%) $\downarrow$
128	-0.416	14.7	72.7	0.0
256	0.146	6.04	84.9	2.4
512	0.246	2.65	88.8	18.1
1024	0.252	1.11	88.9	43.2
2048	0.201	0.48	86.2	58.8

H. Additional Experimental Results

H.1. Baseline Comparison for Varying Data Similarity Mixing Ratio for CV Task

Fig. 13 shows a more comprehensive plot of Fig. 4 for the CV task. We include the unlearning trajectories for more epochs, 25 epochs⁹, compared to 5 epochs in Fig. 4, for $\rho \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$. These results corroborate the conclusions in Fig. 4, where only HAMU performs well in harder unlearning scenarios. We can clearly observe that as the similarity mixing ratio increases, all unlearning methods perform progressively worse, with a smaller ΔL_f and $-\Delta L_r$. Notably, the unlearning performance of the two better performing baselines, GDiff and SCRUB, notably deteriorates beyond $\rho = 0.25$. GDiff is only able to achieve very small magnitude of ΔL_f and $-\Delta L_r$, while for SCRUB, ΔL_f becomes negative. Here, we additionally include the unlearning results for $\rho = 1.0$, where the forget data contains random samples of all classes from the overall training data, with sample distribution indistinguishable from that of the retain data. It can also be noted that when the forget and retain data becomes too similar ($\rho = 1.0$), it becomes impossible to decouple ΔL_f from ΔL_r , regardless of the unlearning algorithm used.

⁹The computation cost of 25 epochs of unlearning is similar to that required for retraining (training was performed for 50 epochs, but on one GPU as opposed to two GPUs for unlearning), which is unrealistic in practice, since a model owner could simply retrain the model from scratch rather than to perform unlearning. Despite this, we show Fig. 13 so as to clearly illustrate the differences between the different unlearning algorithms.

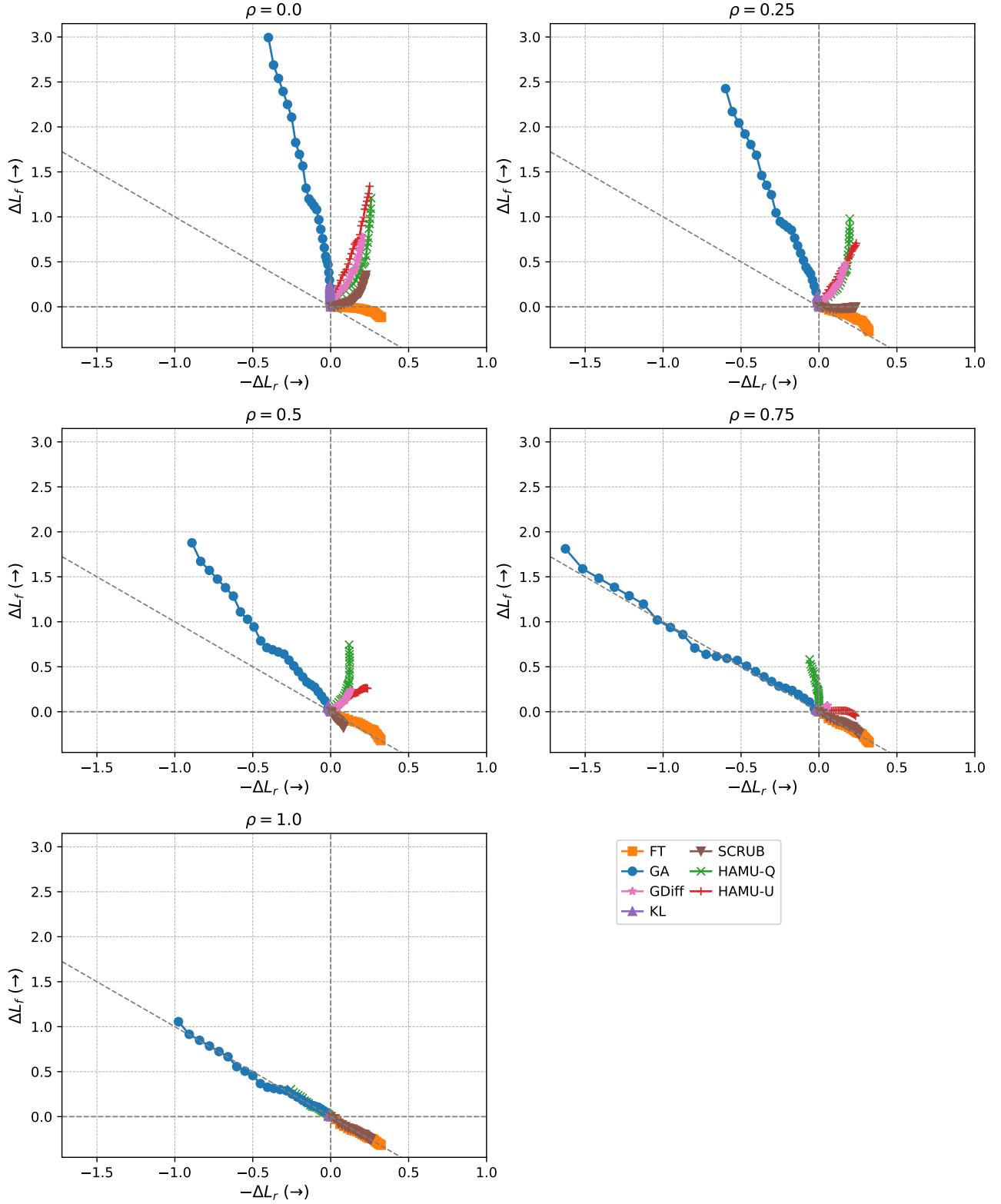


Figure 13. More comprehensive plot of Fig. 4, comparing HAMU with baseline unlearning algorithms for different level of ρ under the CV experimental setting.

H.2. Additional Evaluation Metrics and Baselines for LLM QA Task

We provide results for additional evaluation metrics: token accuracy, model utility (probability) (Maini et al., 2024), and privacy (MIA) (Shi et al., 2024) metrics. We also include two additional baselines, Gradient Rectified Unlearning (GRU) (Wang et al., 2025b) and Projecting Conflicting Gradients (PCGrad) (Yu et al., 2020). PCGrad as described in Yu et al. (2020) performs multi-task optimization and was not initially designed to be used in unlearning. We adapt PCGrad’s update rule to make use of the gradient of the retain data and the negated gradient of the forget data as the task gradients. In this experiment, we focus on the more realistic setting of unlearning with optimizers (described in App. G.2), using learning rate of $1e-5$, and unlearn for 3 epochs.

Table 3 shows that the trend across methods in Retain Acc. and Model Utility follows that of $-\Delta L_r$, and Forget Acc. follows that of ΔL_f . Thus, HAMU achieves a better trade-off and is the only method better than before unlearning on both retain utility and forget quality metrics. HAMU-Q achieves a better MIA (closer to 0.5) as it guarantees sufficient improvement in forget quality unlike HAMU-U and GRU which focus on the retain utility instead.

Table 3. Results for additional baselines and evaluation metrics.

Method	$-\Delta L_r \uparrow$	$\Delta L_f \uparrow$	Retain Acc.(%) $\uparrow$	Forget Acc.(%) $\downarrow$	Model Utility $\uparrow$	MIA
Before unlearning	0.000	0.000	0.924	0.907	0.779	0.999
FT	0.216	-0.104	0.997	0.941	0.966	1.000
GA	-105.392	105.552	0.002	0.002	0.000	0.447
GDiff	-1.452	11.284	0.769	0.485	0.188	0.186
KL	-0.867	7.061	0.805	0.593	0.335	0.098
SCRUB	-0.290	13.179	0.878	0.562	0.575	0.216
GRU	-105.213	105.326	0.002	0.002	0.000	0.457
PCGrad	-1.443	11.325	0.769	0.485	0.189	0.187
HAMU-Q	0.151	2.075	0.976	0.754	0.907	0.552
HAMU-U	0.008	4.372	0.931	0.663	0.788	0.198